



AAC Decoder

Programmer's Guide

For HiFi DSPs



Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134
www.cadence.com

Copyright © 2024 Cadence Design Systems, Inc.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement;
- The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

For further assistance, contact Cadence Online Support at <https://support.cadence.com/>.

Version: 3.11

Last Updated: 12/2024

Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134
www.cadence.com

Contents

Contents	iii
Figures	v
Tables	vi
Document Change History	ix
1. Introduction to the HiFi AAC Decoder	1
1.1 AAC Description	1
1.2 Document Overview	2
1.3 HiFi AAC Decoder Specifications	2
1.4 HiFi AAC Decoder Performance	4
1.4.1 Memory AAC Stereo	4
1.4.2 Timings AAC Stereo	4
1.4.3 Memory aacPlus V1 Stereo	5
1.4.4 Timings aacPlus V1 Stereo	5
1.4.5 Memory aacPlus V2 Stereo	5
1.4.6 Timings aacPlus V2 Stereo	6
1.4.7 Memory AAC Multi-Channel	6
1.4.8 Timings AAC Multi-Channel	6
1.4.9 Memory aacPlus v1 Multi-Channel	7
1.4.10 Timings aacPlus v1 Multi-Channel	7
1.4.11 Memory aacPlus v2 Multi-Channel	7
1.4.12 Timings aacPlus v2 Multi-Channel	8
2. Generic HiFi Audio Codec API	9
2.1 Memory Management	9
2.1.1 API Object	10
2.1.2 API Memory Table	10
2.1.3 Memory Types	10
2.1.3.1 Persistent Memory	10
2.1.3.2 Scratch Memory	10
2.1.3.3 Input Buffer	10
2.1.3.4 Output Buffer	10
2.2 C Language API	11
2.3 About Generic API Errors	12
2.4 Commands	13
2.4.1 Start-up API Stage	14
2.4.2 Set Codec-specific Parameters Stage	14
2.4.3 Memory Allocation Stage	14
2.4.4 Initialize Codec Stage	15

2.4.5	Get Codec-specific Parameters Stage	16
2.4.6	Codec Process Stage	16
2.5	Files Describing the API	17
2.6	HiFi API Command Reference	18
2.6.1	Common API Errors	18
2.6.2	XA_API_CMD_GET_LIB_ID_STRINGS	19
2.6.3	XA_API_CMD_GET_API_SIZE	22
2.6.4	XA_API_CMD_INIT	23
2.6.5	XA_API_CMD_GET_MEMTABS_SIZE	27
2.6.6	XA_API_CMD_SET_MEMTABS_PTR	28
2.6.7	XA_API_CMD_GET_N_MEMTABS	29
2.6.8	XA_API_CMD_GET_MEM_INFO_SIZE	30
2.6.9	XA_API_CMD_GET_MEM_INFO_ALIGNMENT	31
2.6.10	XA_API_CMD_GET_MEM_INFO_TYPE	32
2.6.11	XA_API_CMD_GET_MEM_INFO_PRIORITY	33
2.6.12	XA_API_CMD_SET_MEM_PTR	35
2.6.13	XA_API_CMD_INPUT_OVER	36
2.6.14	XA_API_CMD_SET_INPUT_BYTES	37
2.6.15	XA_API_CMD_GET_CURIDX_INPUT_BUF	38
2.6.16	XA_API_CMD_EXECUTE	39
2.6.17	XA_API_CMD_GET_OUTPUT_BYTES	42
2.6.18	XA_API_CMD_GET_CONFIG_PARAM	43
2.6.19	XA_API_CMD_SET_CONFIG_PARAM	45
3.	HiFi DSP AAC Decoder	46
3.1	AAC Decoder-Specific Files	47
3.2	AAC Decoder-Specific Error Codes	48
3.2.1	API Errors	48
3.2.2	Configuration Errors	49
3.2.3	Process Errors	50
3.3	Configuration Parameters	54
3.3.1	XA_API_CMD_SET_CONFIG_PARAM	56
3.3.2	XA_API_CMD_GET_CONFIG_PARAM	77
3.4	Channel Routing	99
4.	Introduction to the Example Testbench	104
4.1	Making Executable Files	104
4.2	Usage	106
5.	References	109

Figures

Figure 1 HiFi Audio Codec Interfaces 9

Figure 2 API Command Sequence Overview 13

Figure 3 Flow Chart for AAC Decoder Integration 46

Tables

Table 1-1 AAC Stereo Decoder Library Memory	4
Table 1-2 AAC Stereo Decoder Run-Time Memory	4
Table 1-3 AAC Stereo Decoder Timings	4
Table 1-4 aacPlus V1 Stereo Library Memory	5
Table 1-5 aacPlus V1 Stereo Run-Time Memory	5
Table 1-6 aacPlus V1 Stereo Timings	5
Table 1-7 aacPlus V2 Stereo Library Memory	5
Table 1-8 aacPlus V2 Stereo Run-Time Memory	5
Table 1-9 aacPlus V2 Stereo Timings	6
Table 1-10 AAC Multi-Channel Decoder Library Memory	6
Table 1-11 AAC Multi-Channel Decoder Run-Time Memory	6
Table 1-12 AAC Multi-Channel Decoder Timings	6
Table 1-13 aacPlus v1 Multi-Channel Library Memory	7
Table 1-14 aacPlus v1 Multi-Channel Run-Time Memory	7
Table 1-15 aacPlus v1 Multi-Channel Timings	7
Table 1-16 aacPlus v2 Multi-Channel Library Memory	7
Table 1-17 aacPlus v2 Multi-Channel Run-Time Memory	7
Table 2-1 Codec API	11
Table 2-2 Error Codes Format	12
Table 2-3 Commands for Initialization	14
Table 2-4 Commands for Setting Parameters	14
Table 2-5 Commands for Initial Table Allocation	15
Table 2-6 Commands for Memory Allocation	15
Table 2-7 Commands for Initialization	16
Table 2-8 Commands for Getting Parameters	16
Table 2-9 Codec Processing Commands	17
Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand	19
Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand	20
Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand	21
Table 2-13 XA_API_CMD_GET_API_SIZE Command	22
Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand	23
Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand	24
Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand	25
Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand	26
Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command	27
Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command	28
Table 2-20 XA_API_CMD_GET_N_MEMTABS Command	29
Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command	30

Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command.....	31
Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command	32
Table 2-24 Memory Type Indices	32
Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command.....	33
Table 2-26 Memory Priorities	34
Table 2-27 XA_API_CMD_SET_MEM_PTR Command.....	35
Table 2-28 XA_API_CMD_INPUT_OVER Command	36
Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command.....	37
Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command.....	38
Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand.....	39
Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand.....	40
Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand	41
Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command	42
Table 2-35 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand	43
Table 2-36 XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS Subcommand	44
Table 2-37 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand	45
Table 3-1 Library Files (in the lib directory):.....	47
Table 3-2 XA_AACDEC_CONFIG_PARAM_EXTERNALSAMPLINGRATE Subcommand	56
Table 3-3 XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT Subcommand	57
Table 3-4 XA_AACDEC_CONFIG_PARAM_TO_STEREO Subcommand.....	59
Table 3-5 XA_AACDEC_CONFIG_PARAM_PCM_WDSZ Subcommand	60
Table 3-6 XA_AACDEC_CONFIG_PARAM_OUTNCHANS Subcommand.....	61
Table 3-7 XA_AACDEC_CONFIG_PARAM_CHANROUTING Subcommand.....	62
Table 3-8 XA_AACDEC_CONFIG_PARAM_ZERO_UNUSED_CHANS Subcommand.....	64
Table 3-9 XA_AACDEC_CONFIG_PARAM_INPUT_BITOFFSET Subcommand	65
Table 3-10 XA_AACDEC_CONFIG_PARAM_BDOWNSAMPLE Subcommand	66
Table 3-11 XA_AACDEC_CONFIG_PARAM_BBITSTREAMDOWNMIX Subcommand.....	67
Table 3-12 XA_AACDEC_CONFIG_PARAM_SBR_SIGNALING Subcommand.....	68
Table 3-13 XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_PRL Subcommand.....	69
Table 3-14 XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL Subcommand.....	70
Table 3-15 XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_DRC Subcommand.....	71
Table 3-16 XA_AACDEC_CONFIG_PARAM_DRC_COMPRESS_FAC Subcommand.....	72
Table 3-17 XA_AACDEC_CONFIG_PARAM_DRC_BOOST_FAC Subcommand	73
Table 3-18 XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE Subcommand	74
Table 3-19 XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE Subcommand	76
Table 3-20 XA_AACDEC_CONFIG_PARAM_OUT_SAMPLERATE Subcommand.....	77
Table 3-21 XA_AACDEC_CONFIG_PARAM_NUM_CHANNELS subcommand	78
Table 3-22 XA_AACDEC_CONFIG_PARAM_PCM_WDSZ Subcommand	79

Table 3-23	XA_AACDEC_CONFIG_PARAM_SBR_TYPE Subcommand	80
Table 3-24	XA_AACDEC_CONFIG_PARAM_AAC_SAMPLERATE Subcommand	81
Table 3-25	XA_AACDEC_CONFIG_PARAM_DATA_RATE Subcommand	82
Table 3-26	XA_AACDEC_CONFIG_PARAM_CHANMAP Subcommand	83
Table 3-27	XA_AACDEC_CONFIG_PARAM_ACMOD Subcommand	84
Table 3-28	XA_AACDEC_CONFIG_PARAM_AAC_FORMAT Subcommand	85
Table 3-29	XA_AACDEC_CONFIG_PARAM_OUTNCHANS Subcommand	86
Table 3-30	XA_AACDEC_CONFIG_PARAM_DRC_EXT_PRESENT Subcommand	87
Table 3-31	XA_AACDEC_CONFIG_PARAM_MPEG_ID Subcommand	88
Table 3-32	XA_AACDEC_CONFIG_PARAM_ORIGINAL_OR_COPY Subcommand	89
Table 3-33	XA_AACDEC_CONFIG_PARAM_COPYRIGHT_ID_PTR Subcommand	90
Table 3-34	XA_AACDEC_CONFIG_PARAM_PCE_STATUS Subcommand	91
Table 3-35	XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA Subcommand	92
Table 3-36	XA_AACDEC_CONFIG_PARAM_DWNMIX_LEVEL_DVB Subcommand	93
Table 3-37	XA_AACDEC_CONFIG_PARAM_PARSED_DRC_INFO Subcommand	94
Table 3-38	XA_AACDEC_CONFIG_PARAM_PROG_REF_LEVEL Subcommand	95
Table 3-39	XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE Subcommand	96
Table 3-40	XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE Subcommand	97
Table 3-41	XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE Subcommand	98
Table 3-42	Channel Routing Feature Behavior	99

Document Change History

Version	Changes
3.0	- History section added. - Updated Section 1 for LATM support and added memory and timing performance data for HiFi Mini and HiFi 3. - Revised Section 3.
3.1	- Clarified the supported decoding formats in Section 1.3. - Added new API command <code>XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE</code>.
3.2	Added performance data for HiFi 4 in Section 1.4.
3.3	Added LOAS/LATM support variants for all AAC Decoder versions.
3.4	Added performance data for HiFi 3z in Section 1.4.
3.5	- Added support for MPEG4 amendment 4. - Updated channel routing feature. - Added new error codes: <code>XA_AACDEC_CONFIG_NONFATAL_INVALID_PARAM_VALUE</code>, <code>XA_AACDEC_EXECUTE_NONFATAL_ROUTING_ABSENT_CH_IGNORED</code> - Added new API commands <code>XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE</code> and <code>XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE</code> - Updated performance numbers for HiFi 3 with new optimizations.
3.6	Added performance data for Fusion F1 in Section 1.4.
3.7	Added performance data for HiFi 5 in Section 1.4.
3.8	- Added performance data for HiFi 1 in Section 1.4. - Updated in Section 4.2 with an additional command line usage option.
3.9	Added descriptions for the following error codes: <code>XA_AACDEC_CONFIG_NONFATAL_PARTIAL_CHANROUTING</code>

	XA_AACDEC_CONFIG_NONFATAL_INVALID_GEN_STRM_POS XA_AACDEC_CONFIG_NONFATAL_CPID_NOT_PRESENT XA_AACDEC_CONFIG_FATAL_INVALID_EXTERNALBITRATE XA_AACDEC_EXECUTE_NONFATAL_NEXT_SYNC_NOT_FOUND ■ Updated performance numbers.
3.10	■ Updated performance data for RJ-2024.3 toolchain in Section 1.4. ■ Removed Fusion F1 performance data in Section 1.4. ■ Fixed Errata in Table 3-26.
3.11	■ Updated performance data in Section 1.4.

1. Introduction to the HiFi AAC Decoder

The HiFi DSP AAC Decoder is a general term for three different levels of AAC technology supplied by Cadence® Tensilica® as separate libraries:

- The basic library implements the AAC specification recommended by the Moving Picture Experts Group (MPEG), a working group of ISO/IEC.
- The second library, *aacPlus v1 (HE-AAC)*, implements the same AAC functionality with the addition of the SBR (Spectral Band Replication) tool from Coding Technologies.
- The third library, *aacPlus v2 (HE-AAC v2)*, implements everything above and the PS (Parametric Stereo) tool from Coding Technologies.

Cadence Tensilica offers stereo (and 8-channel) versions of each of these three libraries. The stereo version requires less data memory. The LOAS/LATM-supported variant is also available for both the stereo and 8-channel versions.

Note that all eight-channel versions of these three libraries support the extraction of DRC metadata, and the eight-channel HE-AAC-V2 LOAS/LATM-supported library supports the extraction and application of DRC metadata to the output audio.

The rest of this document refers to the HiFi DSP AAC Decoder set of libraries simply as the HiFi AAC Decoder.

The vendor source version is Coding Technologies' *aacPlus Fixed Point Firmware Reference Decoder v4.0.3*.

1.1 AAC Description

AAC (Advanced Audio Coding) was created as part of the MPEG-2 specification ISO/IEC 13818-7. ^[1] It was initially referred to as “non-backward compatible”. This referred to as the move away from compatibility with the existing audio specification (MP3). AAC was then specified and provided with more tools in the MPEG-4 specification ISO/IEC 14496-3. ^[2]

Following is a quote from Coding Technologies with regards to the aacPlus v1 and v2 enhancements:

“MPEG-4 aacPlus is the combination of three MPEG technologies comprising Advanced Audio Coding (AAC), coupled with Coding Technologies’ Spectral Band Replication (SBR) and Parametric Stereo (PS) technologies. SBR is a unique bandwidth extension technique that enables audio codecs to deliver the same quality at half the bit rate. PS significantly increases the codec efficiency a second time for low bit rate stereo signals.” [3] [4]

The target of 1 bit per sample for AAC results in a bitstream of 96kbits/s for stereo 48kHz audio. This is a compression 16 times greater than the uncompressed PCM Audio. With the additional technologies from Coding Technology, this compression increases dramatically.

The MPEG-2 and MPEG-4 specifications mentioned above also extended the AAC audio coding technology to support the encoding and decoding of multi-channel audio streams. Later, the plain AAC was combined with Coding Technologies’ Spectral Band Replication (SBR) technique for encoding multi-channel streams to create multi-channel audio content at very low bit rates of up to 128 kbits/s, while the Parametric Stereo (PS) technology is restricted to stereo content only.

1.2 Document Overview

This document covers all the information required to integrate the HiFi Audio Codecs into an application. The HiFi codec libraries implement a simple API to encapsulate the complexities of the coding operations and simplify the application and system implementation.

- Section 1 describes the HiFi DAB+ decoder in brief.
- Section 2 describes parts of API that are common to all the HiFi codecs.
- Section 3 describes all the features and information particular to the HiFi DAB+ decoder.
- Section 4 describes an example testbench.
- Section 5 lists the references.

1.3 HiFi AAC Decoder Specifications

The HiFi DSP AAC Decoder from Cadence Tensilica implements the following features:

- Uses Cadence Audio Codec API.
- An ISO/IEC 14496 (MPEG-4) compliant decoder supporting the AAC profile (level 5), the HE-AAC profile (level 5), and the HE-AAC v2 profile (level 5), which means the AAC-LC, SBR for up to eight channels, and PS object types are supported.
- A 3GPP TS 26.410 (enhanced aacPlus) compliant decoder.
- The library supports parsing of the following standard formats along with support for decoding raw payloads as follows:

- The ADTS (Audio Data Transport Stream) packets with one raw data block per ADTS frame.
- The ADIF (Audio Data Interchange Format) with one Program Config Element (PCE).
- The LATM (Low Overhead Audio Transport Multiplex) with and without LOAS (Low Overhead Audio Stream). In LOAS, out of three types, only AudioSyncStream() is supported with the restriction of a single program and a single layer with no subframes. This feature is only available for the LOAS-supported libraries.
- The library supports the following sample rates: 8000, 11025, 12000, 16000, 22050, 24000, 32000, 44100, 48000, 64000, 88200 and 96000, in Hz.
- Per MPEG specification, the maximum supported bit rate depends on both the sample rate and the number of channels. At a 48000 Hz sample rate, a 2-channel stream can reach 576 kbps, and an 8-channel stream can reach 2304 kbps. The HiFi AAC Decoder is MPEG-compliant.

There are twelve libraries available for the AAC Decoder, of which six libraries represent the following three levels of AAC profile support, with a stereo-only and an 8-channel version for each level. These libraries support ADTS, ADIF, and raw formats:

- The AAC Decoder library supports the AAC LC profile.
- The aacPlus v1 Decoder library supports the HE-AAC profile, along with the capabilities of the AAC Decoder library.
- The aacPlus v2 Decoder library additionally supports the HE-AAC v2 profile and the capabilities of the aacPlus v1 Decoder.

The additional six libraries represent three levels of AAC profile support mentioned above with LOAS/LATM support and with a stereo-only and an 8-channel version for each level. These libraries support ADTS, ADIF, raw, LOAS, and LATM formats.

Note The following library from the above twelve variants is provided as a full-featured package, which is a superset of the rest of the packages listed above.

- The aacPlus v2 multi-channel with LOAS library. Offered only in the 8-channel package, it supports all features in the aacPlus v2 Decoder library (8-channel), along with LOAS format support and DRC metadata application.

The libraries provide the following user options:

- Arbitrary routing of input channels to interleaved output channels.
- Support for decoding of up to 7.1-channel encoded streams (8-channel libraries only).
- Implicit and explicit SBR signaling (aacPlus libraries only).
- Optional downsampling of SBR output (aacPlus libraries only).
- Optional mono downmix for SBR processing (aacPlus libraries only).
- Partial data feeding is allowed in bitstream formats ADTS and LOAS, that can be enabled through a compile-time switch `SMALL_INPUT_CHUNK`. However, this is not supported in the case of ADIF, raw, or LATM formats.

This decoder implementation has been certified by Coding Technologies. The implementation is based on reference source code^[5].

1.4 HiFi AAC Decoder Performance

The Cadence Tensilica's HiFi DSP AAC Decoders are characterized on the HiFi 5-stage DSP. The memory usage and performance figures are provided for design reference.

- The API structure sizes returned by XA_API_CMD_GET_API_SIZE range from 180 to 220 bytes, depending on which particular library is used.
- The memory table structure size returned by XA_API_CMD_GET_MEMTABS_SIZE is approximately 150 bytes.

1.4.1 Memory AAC Stereo

Table 1-1 AAC Stereo Decoder Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	43.1	41.0	44.3	45.4	50.9	26.9
Yes	46.3	44.3	47.4	48.7	54.1	27.0

Table 1-2 AAC Stereo Decoder Run-Time Memory

LOAS Support	Runtime Memory (Kbytes)				
	Persistent	Scratch	Stack	Input	Output
No	5.5	12.1	1.2	1.8	8.0
Yes	6.2	12.1	1.2	8.3	8.0

Note The output buffer requirement is halved if the output PCM size is 16 bits.

1.4.2 Timings AAC Stereo

Table 1-3 AAC Stereo Decoder Timings

Rate (kHz)	Channels	Bit Rate (Kbps)	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
44.1	2	96	4.1	4.3	3.6	3.4	3.3
48	2	128	4.6	4.9	4.0	3.8	3.7
48	2	320	5.5	5.9	4.9	4.6	4.5

1.4.3 Memory aacPlus V1 Stereo

Table 1-4 aacPlus V1 Stereo Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	74.4	70.7	75.4	76.3	84.9	37.9
Yes	77.7	73.9	78.6	79.6	88.2	37.9

Table 1-5 aacPlus V1 Stereo Run-Time Memory

LOAS Support	Runtime Memory (Kbytes)				
	Persistent	Scratch	Stack	Input	Output
No	24.3	22.2	1.7	1.8	16.0
Yes	25.1	22.2	1.7	8.3	16.0

Note In this case, the output buffer requirement is the same for both 16-bit and 24-bit PCM data.

1.4.4 Timings aacPlus V1 Stereo

Table 1-6 aacPlus V1 Stereo Timings

Rate (kHz)	Channels	Bit Rate (Kbps)	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
44.1	2	64	16.5	14.8	12.8	11.0	11.0
48	2	128	18.0	16.3	14.1	12.2	12.1

1.4.5 Memory aacPlus V2 Stereo

Table 1-7 aacPlus V2 Stereo Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	83.6	80.0	85.5	87.2	96.2	39.8
Yes	86.8	83.2	88.8	90.4	99.5	39.9

Table 1-8 aacPlus V2 Stereo Run-Time Memory

LOAS Support	Runtime Memory (Kbytes)				
	Persistent	Scratch	Stack	Input	Output
No	28.9	22.2	1.8	1.8	16.0
Yes	29.7	22.2	1.8	8.3	16.0

Note In this case, the output buffer requirement is the same for both 16-bit and 24-bit PCM data.

1.4.6 Timings aacPlus V2 Stereo

Table 1-9 aacPlus V2 Stereo Timings

Rate (kHz)	Channels	Bit Rate (Kbps)	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
44.1	2	48	17.5	15.9	13.9	12.2	12.1
48	2	64	18.7	17.0	14.9	13.0	12.9

1.4.7 Memory AAC Multi-Channel

Table 1-10 AAC Multi-Channel Decoder Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	45.5	43.3	46.8	47.9	53.5	27.0
Yes	48.8	46.6	50.1	51.2	56.7	27.0

Table 1-11 AAC Multi-Channel Decoder Run-Time Memory

LOAS Support	Channels	Runtime Memory (Kbytes)				
		Persistent	Scratch	Stack	Input	Output
No	6	14.9	35.4	1.2	4.8	24.0
No	8	19.1	47.2	1.2	6.3	32.0
Yes	6	15.7	45.4	1.2	8.3	24.0
Yes	8	19.9	47.2	1.2	8.3	32.0

Note The output buffer requirement is halved if the output PCM size is 16 bits.

1.4.8 Timings AAC Multi-Channel

Table 1-12 AAC Multi-Channel Decoder Timings

Rate (kHz)	Channels	Bit Rate (Kbps)	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
48	6	384	10.0	9.4	8.1	7.4	7.2
48	8	576	16.1	16.7	14.0	12.8	12.1

1.4.9 Memory aacPlus v1 Multi-Channel

Table 1-13 aacPlus v1 Multi-Channel Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	77.0	73.3	78.0	78.9	87.6	38.0
Yes	80.3	76.5	81.3	82.2	90.9	38.0

Table 1-14 aacPlus v1 Multi-Channel Run-Time Memory

LOAS Support	Channels	Runtime Memory (Kbytes)				
		Persistent	Scratch	Stack	Input	Output
No	6	71.5	35.4	1.7	4.8	48.0
No	8	94.5	47.2	1.7	6.3	64.0
Yes	6	72.3	35.4	1.7	8.3	48.0
Yes	8	95.3	47.2	1.7	8.3	64.0

Note In this case, the output buffer requirement is the same for 16-bit and 24-bit PCM data.

1.4.10 Timings aacPlus v1 Multi-Channel

Table 1-15 aacPlus v1 Multi-Channel Timings

Rate (kHz)	Channels	Bit Rate (Kbps)	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
48	6	128	46.5	40.3	35.7	30.2	30.0
48	8	192	61.4	53.4	47.4	39.9	39.7

1.4.11 Memory aacPlus v2 Multi-Channel

Table 1-16 aacPlus v2 Multi-Channel Library Memory

LOAS Support	Text (Kbytes)					Data (Kbytes)
	HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
No	86.2	82.6	88.2	89.8	98.9	39.9
Yes	93.7	89.9	95.8	97.5	107.1	40.1

Table 1-17 aacPlus v2 Multi-Channel Run-Time Memory

LOAS Support	Channels	Runtime Memory (Kbytes)				
		Persistent	Scratch	Stack	Input	Output
No	6	73.9	35.4	1.7	4.8	48.0
No	8	96.9	47.2	1.7	6.3	64.0
Yes*	6	75.9	35.4	1.8	8.3	48.0
Yes*	8	98.9	47.2	1.8	8.3	64.0

Note In this case, the output buffer requirement is the same for 16-bit and 24-bit PCM data.

Note * This library also supports the application of DRC metadata embedded in the stream.

1.4.12 Timings aacPlus v2 Multi-Channel

The timing specifications of the aacPlus v2 multi-channel library are identical to those of the aacPlus v1 multi-channel library for AAC and aacPlus v1 streams and to those of the aacPlus v2 stereo library for aacPlus v2 streams.

Note Performance specification measurements are carried out on a cycle-accurate simulator assuming an ideal memory system, *i.e.*, one with zero memory wait states. This is equivalent to running with all the code and the data in local memories or using an infinite-size, pre-filled cache model.

Note The MCPS numbers for HiFi1/HiFi 3/HiFi 3z/HiFi 4/HiFi 5 are obtained by running the test with binaries based on 24-bit optimized source code.

Note Except for AAC Stereo (aac_dec), no specific optimization is performed for HiFi 3/HiFi 3z/HiFi 4/HiFi 5/HiFi1.

Note For the AAC Stereo library, specific optimization is performed for HiFi 3, and the MCPS numbers for HiFi 3z/HiFi 4/HiFi 5/Fusion F1 are obtained by running the test that is recompiled from the HiFi 3 source code in the HiFi 3z/HiFi 4/HiFi 5/Fusion F1 configuration.

Note The HiFi3 optimized code is further tuned specifically for HiFi1, and MCPS numbers for HiFi1 are thus obtained by compiling this optimized code.

Note The increase in Average CPU Load (MHz) for LOAS/LATM processing is negligible (<0.1 MHz).

Note All the memory and MCPS numbers are captured with RJ-2024.3 tools and the XTCLANG compiler.

2. Generic HiFi Audio Codec API

This section provides information about the API that is common to all the HiFi audio codec libraries. The API facilitates any codec that works in the overall method shown in Figure 1.

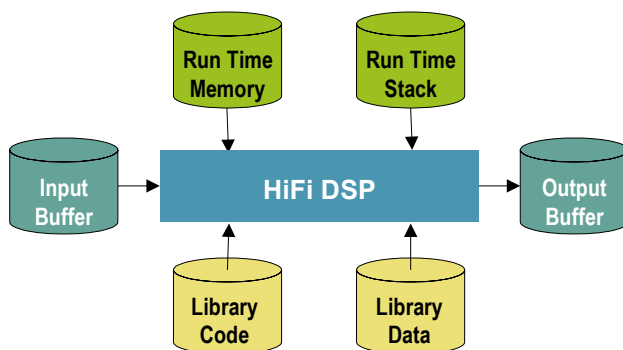


Figure 1 HiFi Audio Codec Interfaces

Section 2.1 discusses all the types of run-time memory required by the codecs. As no state information is held in static memory, a single thread can perform time division processing of multiple codecs. Additionally, multiple threads can perform concurrent codec processing. The API is implemented so that the application does not need to consider the codec implementation.

The codec requests the minimum sizes required for the input and output buffers through the API. Hence, before running the codec command, the codec requires that the input buffer is filled with data up to the minimum size for the input buffer. However, the codec may not consume all of the data in the input buffer. To properly manage the input data, the application needs to monitor the amount of data used, transfer any remaining unused data to the lower part of the input buffer, and then proceed to fill the remaining space in the buffer with new data until it reaches the minimum size again. The codec produces data in the output buffer. After the codec operation, the output data must be removed from the output buffer.

Applications that use these libraries should not make any assumptions about the size of the PCM “chunks” of data that each call to a codec produces or consumes. Normally, the chunks are the exact size of the underlying frame of the specified codec algorithm, and they will vary between codecs and different operating modes of the same codec. The application should provide enough data to fill the input buffer. However, some codecs provide information after the initialization stage to adjust the number of bytes of PCM data they need.

2.1 Memory Management

The HiFi audio codec API supports a flexible memory scheme and a simple interface that simplifies the integration into the final application. The API allows the codecs to request the memory required for their operations during run time.

The run-time memory requirement consists primarily of scratch and persistent memory. The codecs also require an input buffer and output buffer to transfer data into and out of the codec.

2.1.1 API Object

For each API call, the codec API stores its data in a small structure passed via a pointer handle to an opaque object from the application. The codec references all state information and the memory tables from this structure.

2.1.2 API Memory Table

During the memory allocation, the application is prompted to allocate memory for each of the following memory areas (described in Section 2.1.3). The reference pointer to each memory area is stored in this memory table, and the reference to this memory table is stored in the API object.

2.1.3 Memory Types

2.1.3.1 Persistent Memory

This type of memory is referred to as static or context memory. It consists of the state or historical information preserved from one codec invocation to the next within the same thread or instance. The codecs assume that the persistent memory remains unchanged by the system, except for the codec library itself, for the entire duration of the codec operation.

2.1.3.2 Scratch Memory

The codec uses this type of temporary buffer for processing. The contents of this memory region should be unchanged if the codec processing is active, that is if the thread running the codec is inside any API call. The system can use this region freely between successive calls to the codec.

2.1.3.3 Input Buffer

This is the buffer that the algorithm uses to accept input data. Before the call to the codec, the input buffer must be completely filled with input data.

2.1.3.4 Output Buffer

This output buffer is used by algorithm to write the output. Before the algorithm completes its tasks, the output buffer must be accessible for the codec. The application can modify the output buffer pointer between codec calls, enabling the codec to write directly to the designated output area. It's important to note that the codec will never write more data than the requested size of the output buffer.

2.2 C Language API

A single interface function is used to access the codec, with the operation specified by command codes. The API C call is defined for each codec library and is specified in the codec-specific section. Each library has a single C API call.

The C parameter definitions for every codec library are the same and are specified in the following table:

Table 2-1 Codec API

xa_<codec>	
Description	This C API is the only function that allows accessing the audio codec.
Syntax	<pre>XA_ERRORCODE xa_<codec>(xa_codec_handle_t p_xa_module_obj, WORD32 i_cmd, WORD32 i_idx, pVOID pv_value);</pre>
Parameters	<code>p_xa_module_obj</code> Pointer to the opaque API structure. <code>i_cmd</code> Command. <code>i_idx</code> Command subtype or index. <code>pv_value</code> Pointer to the variable used to pass in or retrieve properties from the state structure.
Returns	Returns an error code to indicate the success or failure of the API call.

The types used for the C API calls are defined in the supplied header files as:

```
typedef signed int      WORD32;
typedef void            *pVOID;
```

Each time the C API for the codec is called, a pointer to a privately allocated data structure is passed as the first argument. This argument is treated as an opaque handle, as the application does not require the data to be looked at within the structure. The size of the structure is supplied by a specific API command so that the application can allocate the required memory. Do not use `sizeof()` for the type of the opaque handle.

Some command codes are further divided into subcommands. The command and its subcommand are passed to the codec via the second and third arguments, respectively.

When a value is passed to or returned from an API command, the expected or returned value is passed through a pointer given as the fourth argument to the C API function. When passing a pointer value to the codec, the pointer is cast to `pVOID`. In these cases, passing a pointer to a pointer is incorrect. For example, when the application passes the codec a pointer to an allocated memory region.

Because the operations required to decode or encode audio streams are similar, the HiFi DSP API enables the application to use the same set of procedures for each stage. By maintaining a pointer to the single API function and passing the correct API object, the same code base can be used to carry out the operations required for any of the supported codecs.

2.3 About Generic API Errors

The API functions return the error code of `XA_ERRORCODE` type, which is `signed int`. Table 2-2 defines the format of the error codes.

Table 2-2 Error Codes Format

31	30-15	14 - 11	10 - 6	5 - 0
Fatal	Reserved	Class	Codec	Sub code

The format of error codes is explained as follows:

- The errors returned from the APIs are divided into fatal and nonfatal errors based on severity. Fatal errors require restarting the codec, while nonfatal errors provide information to the application.
- Errors can be further classified into the following three classes:
 - API errors occur when the API is used incorrectly.
 - Config errors occur when the codec parameters are incorrect or not supported.
 - Process Errors occur during the primary encoding or decoding and indicate issues arising due to invalid or bad input data.
- The Sub code specifies the exact condition that triggers the error.

2.4 Commands

This section details the commands associated with the flow chart shown in Figure 2. The white blocks in the flow chart represent a common set of generic API commands, and the yellow blocks represent codec-specific commands.

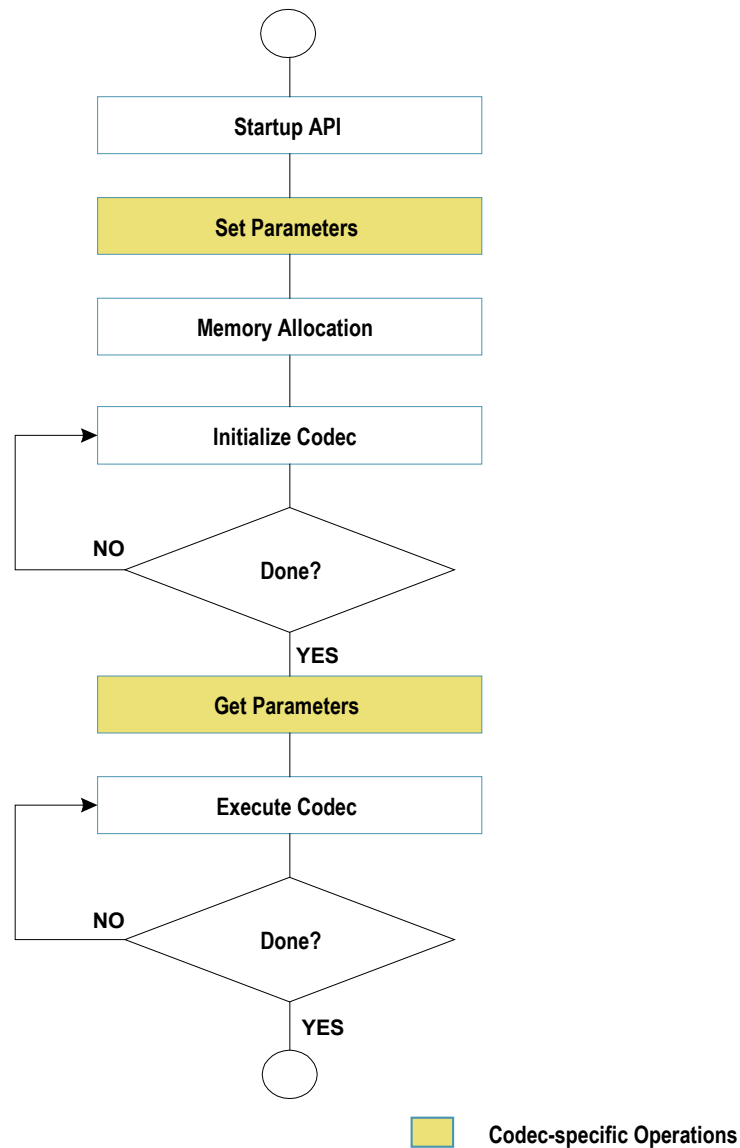


Figure 2 API Command Sequence Overview

The following sections list the commands in the order they should occur for each stage of the flow shown in Figure 2. For individual commands, definitions, and examples, refer to Section 2.6.

2.4.1 Start-up API Stage

The commands listed in Table 2-3 should be run once each during start-up. The commands to get the various identification strings from the codec library are optional and for information only. The command to get the API object size is mandatory as the real object type is hidden in the library, and therefore there is no type available to use with `sizeof()`.

Table 2-3 Commands for Initialization

Command / Subcommand	Description
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_NAME	Gets the name of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_VERSION	Gets the version of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_API_VERSION	Gets the version of the API.
XA_API_CMD_GET_API_SIZE	Gets the size of the API structure.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS	Sets the default values of all the configuration parameters.

2.4.2 Set Codec-specific Parameters Stage

Refer to the specific codec section for the parameters that can be set. These parameters either control the encoding process or determine the output format of the decoder PCM data.

Table 2-4 Commands for Setting Parameters

Command / Subcommand	Description
XA_API_CMD_SET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Sets codec-specific parameters. See the codec-specific section for parameter definitions.

2.4.3 Memory Allocation Stage

The commands listed in Table 2-5 should be run once, only after all the codec-specific parameters are set. After the application allocates the memory table structure (MEMTABS) per the specified size, a pointer to the memory table structure is passed to the codec API. After the codec-specific parameters are set, the initial codec setup is completed by performing the post-configuration portion of the initialization to determine the initial operating mode of the codec and assign sizes to the blocks of memory required for its operation. The application then requests a count of the number of memory blocks.

Table 2-5 Commands for Initial Table Allocation

Command / Subcommand	Description
<code>XA_API_CMD_GET_MEMTABS_SIZE</code>	Gets the size of the memory structures to be allocated for the codec tables.
<code>XA_API_CMD_SET_MEMTABS_PTR</code>	Passes the memory structure pointer allocated for the tables.
<code>XA_API_CMD_INIT</code> <code>XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS</code>	Calculates the required sizes for all the memory blocks based on the codec-specific parameters.
<code>XA_API_CMD_GET_N_MEMTABS</code>	Obtains the number of memory blocks required by the codec.

The commands listed in Table 2-6 should then be run in a loop to allocate the memory. The application first requests all the attributes of the memory block and then allocates it, making sure to abide by the alignment requirements. Finally, the pointer to the allocated block of memory is passed back through the API. For the input and output buffers, it is not necessary to assign the correct memory at this point. However, the input and output buffer locations must be assigned before their first use in the Process stage. The type field refers to the memory blocks, such as input or persistent, as described in Section 2.1.

Table 2-6 Commands for Memory Allocation

Command / Subcommand	Description
<code>XA_API_CMD_GET_MEM_INFO_SIZE</code>	Gets the size of the memory type being referred to by the index.
<code>XA_API_CMD_GET_MEM_INFO_ALIGNMENT</code>	Gets the alignment information of the memory type being referred to by the index.
<code>XA_API_CMD_GET_MEM_INFO_TYPE</code>	Gets the type of memory referenced by the index.
<code>XA_API_CMD_GET_MEM_INFO_PRIORITY</code>	Gets the allocation priority of memory being referred to by the index.
<code>XA_API_CMD_SET_MEM_PTR</code>	Sets the pointer to the memory allocated for the referred index to the input value.

2.4.4 Initialize Codec Stage

The commands listed in Table 2-7 should be run in a loop during initialization. These commands should be called until the initialization is completed, as indicated by the `XA_CMD_TYPE_INIT_DONE_QUERY` command. In general, decoders can loop multiple times until the header information is found. However, encoders will perform exactly one call before they signal they are done.

There is a major difference between encoding Pulse Code Modulated (PCM) data and decoding stream data. During the initialization of a decoder, the initialization task reads the input stream to discover the encoding parameters. However, for an encoder, PCM data has no header information. Even so, the encoder application is still required to perform the initialization described in this stage. However, encoders will not

consume data during initialization. Furthermore, this implies that some encoders provide parameters that can be used to modify the input buffer data requirements after the initialization stage. These modifications will always be a reduction in size. The application only needs to provide the reduced amount for each of the main codec processes.

In general, the application will signal the codec the number of bytes available in the input buffer and signal whether it is the last iteration. It is not normal to reach the end of the data during initialization. Still, if a decoder is presented with a corrupt stream, it will allow a smooth termination. After the codec initialization is called, the application will ask for the number of bytes consumed. The application can also ask if the initialization is complete. It is advisable to always ask, even in encoders requiring only a single pass. A decoder application must keep iterating until the initialization is completed.

Table 2-7 Commands for Initialization

Command / Subcommand	Description
XA_API_CMD_SET_INPUT_BYTES	Sets the number of bytes available in the input buffer for initialization.
XA_API_CMD_INPUT_OVER	Signals the codec at the end of the bitstream.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_PROCESS	Searches for the valid header, decodes header to get the parameters, and initializes state and configuration structures.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_DONE_QUERY	Checks if the initialization process has been completed.
XA_API_CMD_GET_CURIDX_INPUT_BUF	Gets the number of input buffer bytes consumed by the last initialization.

2.4.5 Get Codec-specific Parameters Stage

After initialization, the codec can supply the application with information. In the case of decoders, these would be the parameters it has extracted from the encoded header in the stream.

Table 2-8 Commands for Getting Parameters

Command / Subcommand	Description
XA_API_CMD_GET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Gets the parameter value from the codec. See the codec-specific section for parameter definitions.

2.4.6 Codec Process Stage

The commands listed in Table 2-9 should be run continuously until the data is exhausted or the application wants to terminate the process. This is similar to the initialization stage but includes support for the management of the output buffer. After each iteration, the application requests the amount of data written to the output buffer. This amount is always limited by the size of the requested buffer during the memory block allocation. To alter the output buffer position, use XA_API_CMD_SET_MEM_PTR with the output buffer index.

Table 2-9 Codec Processing Commands

Command / Subcommand	Description
XA_API_CMD_INPUT_OVER	Signals the end of the bitstream to the library.
XA_API_CMD_SET_INPUT_BYTES	Sets the number of bytes available in the input buffer for the codec processing.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DO_EXECUTE	Runs the codec thread.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DONE_QUERY	Checks if the end of the stream has been reached.
XA_API_CMD_GET_OUTPUT_BYTES	Gets the number of bytes output by the codec in the last frame.
XA_API_CMD_GET_CURIDX_INPUT_BUF	Gets the number of input buffer bytes consumed by the last call to the codec.

2.5 Files Describing the API

Following are the common include files (include):

- `xa_apicmd_standards.h`
Includes command definitions for the generic API calls.
- `xa_error_standards.h`
Includes macros and definitions for all the generic errors.
- `xa_memory_standards.h`
Includes definitions for memory block allocation.
- `xa_type_def.h`
Includes definitions of all datatypes required for the API calls.

2.6 HiFi API Command Reference

This section describes different commands along with their associated subcommands. This section does not describe the codec-specific commands. The particular codec commands are the SET and GET commands for the operational parameters.

The commands are listed below in the following sections based on their primary command type (`i_cmd`). Each section contains a table for every subcommand. If subcommands are not available, one primary command is presented.

The commands are followed by an example C call code. Along with the call, a definition of the variable types used is provided. This is to avoid any confusion over the type of the fourth argument. The examples are not complete C code extracts, as the variables are not initialized before they are used.

The errors returned by the API are listed after each of the command definitions. However, a few errors are common to all the API commands; these are listed in Section 2.6.1. The codec-specific sections will define all the errors possible from the codec-specific commands. For more information on Process errors, refer to Section 3.2.3.

2.6.1 Common API Errors

These errors are fatal and should not be encountered during normal application operations. They signal that a serious error has occurred in the application, calling the codec.

- `XA_API_FATAL_MEM_ALLOC`
`p_xa_module_obj` is NULL.
- `XA_API_FATAL_MEM_ALIGN`
`p_xa_module_obj` is not aligned to 4 bytes.
- `XA_API_FATAL_INVALID_CMD`
`i_cmd` is not a valid command.
- `XA_API_FATAL_INVALID_CMD_TYPE`
`i_idx` is invalid for the specified command (`i_cmd`).

2.6.2 XA_API_CMD_GET_LIB_ID_STRINGS

Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand

Subcommand	XA_CMD_TYPE_LIB_NAME
Description	This command obtains the library's name in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer with a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_NAME pv_value process name - Pointer to a character buffer in which the library name is returned.
Restrictions	None

Note Codec object is not required as the name is static data in the codec library.

Example

```
char process_name[30];
res = (*api_func) (NULL,
    XA_API_CMD_GET_LIB_ID_STRINGS,
    XA_CMD_TYPE_LIB_NAME,
    (pVOID) process_name);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand

Subcommand	XA_CMD_TYPE_LIB_VERSION
Description	This command obtains the library version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_VERSION pv_value lib_version - Pointer to a character buffer in which the version of the library is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char lib_version[30];
res = (*api_func) (NULL,
                  XA_API_CMD_GET_LIB_ID_STRINGS,
                  XA_CMD_TYPE_LIB_VERSION,
                  (pVOID) lib_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand

Subcommand	XA_CMD_TYPE_API_VERSION
Description	This command obtains the API version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer with a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_API_VERSION pv_value api_version - Pointer to a character buffer in which the version of the API is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char api_version[30];
res = (*api_func) (NULL,
                  XA_API_CMD_GET_LIB_ID_STRINGS,
                  XA_CMD_TYPE_API_VERSION,
                  (pVOID) api_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.3 XA_API_CMD_GET_API_SIZE

Table 2-13 XA_API_CMD_GET_API_SIZE Command

Subcommand	None
Description	This command is used to obtain the size of the API structure and allocate memory for the API structure. The pointer to the API size variable is passed and the API returns the structure size in bytes. The API structure is used for the interface and is persistent.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_API_SIZE i_idx NULL pv_value &api_size – Pointer to the API size variable.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Note Codec object is not required as the size is fixed for the codec library.

Example

```
unsigned int api_size;
res = (*api_func) (NULL,
    XA_API_CMD_GET_API_SIZE,
    0,
    (pVOID) &api_size);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.4 XA_API_CMD_INIT

Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS
Description	This command sets the default value of the configuration parameters. The configuration parameters can then be altered using one of the codec-specific parameter setting commands. Refer to Section 3.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
                XA_API_CMD_INIT,  
                XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS,  
                NULL);
```

Errors

- Common API Errors.

Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS
Description	This command calculates the sizes of all the memory blocks required by the application. It should occur after the codec-specific parameters are set.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
    XA_API_CMD_INIT,  
    XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS,  
    NULL);
```

Errors

- Common API Errors.

Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand

Subcommand	XA_CMD_TYPE_INIT_PROCESS
Description	This command initializes the codec. In the case of a decoder, it searches for the valid header and performs the header decoding to get the encoded stream parameters. This command is part of the initialization loop. It must be repeatedly called until the codec signals it has finished. In the case of an encoder, the initialization of the codec is performed. No output data is created during initialization.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_PROCESS pv_value NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_INIT,  
                  XA_CMD_TYPE_INIT_PROCESS,  
                  NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process Errors.

Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_INIT_DONE_QUERY
Description	This command checks to see if the initialization process has been completed. If it has, the flag value is set to 1; otherwise, it is set to zero. A pointer to the flag variable is passed as an argument.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_DONE_QUERY pv_value &init_done – Pointer to a flag that indicates the completion of the initialization process.
Restrictions	None

Example

```

unsigned int init_done;
res = (*api_func) (api_obj,
                  XA_API_CMD_INIT,
                  XA_CMD_TYPE_INIT_DONE_QUERY,
                  (pVOID) &init_done);

```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.5 XA_API_CMD_GET_MEMTABS_SIZE

Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command

Subcommand	None
Description	This command obtains the table size to hold the memory blocks required for the codec operation. The API returns the total size of the required table. A pointer to the size variable is sent with this API command, and the codec writes the variable's value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEMTABS_SIZE i_idx NULL pv_value &proc_mem_tabs_size – Pointer to the memory size variable.
Restrictions	The application shall allocate memory with an alignment of 4 bytes.

Example

```

unsigned int proc_mem_tabs_size;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_MEMTABS_SIZE,
                  0,
                  (pVOID) &proc_mem_tabs_size);

```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.6 XA_API_CMD_SET_MEMTABS_PTR

Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command

Subcommand	None
Description	This command sets the memory structure pointer in the library to the allocated value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEMTABS_PTR i_idx NULL pv_value alloc – Allocated pointer.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Example

```
int * alloc; //alloc is a pointer to the allocated memory
res = (*api_func)(api_obj,
    XA_API_CMD_SET_MEMTABS_PTR,
    0,
    (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_MEM_ALIGN
pv_value is not aligned to 4 bytes.

2.6.7 XA_API_CMD_GET_N_MEMTABS

Table 2-20 XA_API_CMD_GET_N_MEMTABS Command

Subcommand	None
Description	This command obtains the number of memory blocks needed by the codec. This value is used as the iteration counter for allocating the memory blocks. A pointer to each memory block will be placed in the previously allocated memory tables. The pointer to the variable is passed to the API, and the codec writes the value for this variable.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_N_MEMTABS i_idx NULL pv_value &n_mems – Number of memory blocks required to be allocated.
Restrictions	None

Example

```
int n_mems;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_N_MEMTABS,
                  0,
                  (pVOID) &n_mems);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.8 XA_API_CMD_GET_MEM_INFO_SIZE

Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command

Subcommand	Memory index
Description	This command obtains the size of the memory type referred to by the index. The size in bytes is returned in the variable pointed to by the final argument. Note that this is the actual size needed and does not include any alignment packing space.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_SIZE i_idx Index of the memory pv_value &size – Pointer to the memory size.
Restrictions	None

Example

```
int index;
unsigned int size;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_MEM_INFO_SIZE,
                  index,
                  (pVOID) &size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.9 XA_API_CMD_GET_MEM_INFO_ALIGNMENT

Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command

Subcommand	Memory index
Description	This command gets the alignment information of the memory type referred to by the index. The alignment required in bytes is returned to the application.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_ALIGNMENT i_idx Index of the memory. pv_value &alignment – Pointer to the alignment info variable
Restrictions	None

Example

```
int index;
unsigned int alignment;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_ALIGNMENT,
    index,
    (pVOID) &alignment);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.10 XA_API_CMD_GET_MEM_INFO_TYPE

Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command

Subcommand	Memory index
Description	This command gets the type of memory being referred to by the index.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_TYPE i_idx Index of the memory. pv_value &type – Pointer to the memory type variable.
Restrictions	None

Example

```
int index;
unsigned int type;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_TYPE,
    index,
    (pVOID) &type);
```

Table 2-24 Memory Type Indices

Type	Description
XA_MEMTYPE_PERSIST	Persistent Memory
XA_MEMTYPE_SCRATCH	Scratch Memory
XA_MEMTYPE_INPUT	Input Buffer
XA_MEMTYPE_OUTPUT	Output Buffer

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.11 XA_API_CMD_GET_MEM_INFO_PRIORITY

Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command

Subcommand	Memory index
Description	This command gets the allocation priority of memory referred to by the index. (The meaning of the levels is defined on a codec-specific basis. Unless the codec defines it otherwise, this command returns a fixed dummy value.)
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_PRIORITY i_idx Index of the memory. pv_value &priority – Pointer to the memory priority variable.
Restrictions	None

Example

```
int index;
unsigned int priority;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_MEM_INFO_PRIORITY,
                  index,
                  (pVOID) &priority);
```

Table 2-26 Memory Priorities

Priority	Type
0	XA_MEMPRIORITY_ANYWHERE
1	XA_MEMPRIORITY_LOWEST
2	XA_MEMPRIORITY_LOW
3	XA_MEMPRIORITY_NORM
4	XA_MEMPRIORITY_ABOVE_NORM
5	XA_MEMPRIORITY_HIGH
6	XA_MEMPRIORITY_HIGHER
7	XA_MEMPRIORITY_CRITICAL

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.12 XA_API_CMD_SET_MEM_PTR

Table 2-27 XA_API_CMD_SET_MEM_PTR Command

Subcommand	Memory index
Description	This command passes the pointer to the allocated memory to the codec. The pointer is then stored in the memory tables structure allocated earlier. For the input and output buffers, running this command during the main codec loop is legitimate.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEM_PTR i_idx Index of the memory. pv_value alloc – Pointer to the allocated memory buffer.
Restrictions	The pointer must be correctly aligned to the requirements.

Example

```
int index;
void * alloc; //alloc is a pointer to the aligned memory
res = (*api_func)(api_obj,
                 XA_API_CMD_SET_MEM_PTR,
                 index,
                 (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).
- XA_API_FATAL_MEM_ALIGN
pv_value is not of the required alignment for the requested memory block.

2.6.13 XA_API_CMD_INPUT_OVER

Table 2-28 XA_API_CMD_INPUT_OVER Command

Subcommand	None
Description	This command informs the codec that the end of the input data has been reached. This situation can arise both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INPUT_OVER i_idx NULL pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
    XA_API_CMD_INPUT_OVER,  
    0,  
    NULL);
```

Errors

- Common API Errors.

2.6.14 XA_API_CMD_SET_INPUT_BYTES

Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command

Subcommand	None
Description	This command sets the number of bytes available in the input buffer for the codec. It is used both in the initialization and processing loop. It is the number of valid bytes from the buffer pointer. At least the minimum buffer size should be requested unless this is the end of the data.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_INPUT_BYTES i_idx NULL pv_value &buff_size – Pointer to the input byte variable.
Restrictions	None

Example

```
int buff_size;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_INPUT_BYTES,
                  0,
                  (pVOID) &buff_size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.15 XA_API_CMD_GET_CURIDX_INPUT_BUF

Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command

Subcommand	None
Description	This command gets the number of input buffer bytes consumed by the codec. It is used both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CURIDX_INPUT_BUF i_idx NULL pv_value &bytes_consumed – Pointer to the bytes consumed variable.
Restrictions	None

Example

```
int bytes_consumed;  
res = (*api_func)(api_obj,  
    XA_API_CMD_GET_CURIDX_INPUT_BUF,  
    0,  
    (pVOID) &bytes_consumed);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.16 XA_API_CMD_EXECUTE

Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand

Subcommand	XA_CMD_TYPE_DO_EXECUTE
Description	This command runs the codec.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DO_EXECUTE pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
                 XA_API_CMD_EXECUTE,  
                 XA_CMD_TYPE_DO_EXECUTE,  
                 NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process errors.

Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_DONE_QUERY
Description	This command checks to see if the processing has been completed. If it has, the flag value is set to 1; otherwise, it is set to zero. The pointer to the flag is passed as an argument. Processing by the codec can continue for several invocations of the DO_EXECUTE command after the last input data has been passed to the codec. Thus, the application should not assume that the codec has finished generating all its output until the command indicates it.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DONE_QUERY pv_value &flag – Pointer to the flag variable.
Restrictions	None

Example

```
int flag;  
res = (*api_func)(api_obj,  
                 XA_API_CMD_EXECUTE,  
                 XA_CMD_TYPE_DONE_QUERY,  
                 (pVOID) &flag);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand

Subcommand	XA_CMD_TYPE_DO_RUNTIME_INIT
Description	This command resets the decoder's history buffers. It can be used to avoid distortions and clicks by facilitating playback ramping up and down during trick play. The command should be issued before the application feeds the decoder with new data from a random place in the input stream. Note: This command is available in API version 1.14 or later.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DO_RUNTIME_INIT pv_value NULL
Restrictions	None.

Example

```
res = (*api_func)(api_obj,  
                  XA_API_CMD_EXECUTE,  
                  XA_CMD_TYPE_DO_RUNTIME_INIT,  
                  NULL);
```

Errors

- Common API Errors.

2.6.17 XA_API_CMD_GET_OUTPUT_BYTES

Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command

Subcommand	None
Description	This command obtains the number of bytes output during the last codec processing.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_OUTPUT_BYTES i_idx NULL pv_value &out_bytes – Pointer to the output bytes variable.
Restrictions	None

Example

```
int out_bytes;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_OUTPUT_BYTES,
    0,
    (pVOID) &out_bytes);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.18 XA_API_CMD_GET_CONFIG_PARAM

Table 2-35 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS
Description	This command reads the current input stream position, which is equal to the total number of consumed input bytes until the start of the input buffer. This running counter is set to zero at library initialization time and incremented every time the codec library consumes any bytes from the input buffer. If the application layer places a unit of input data with a byte size equal to <i>size</i> at byte offset in the input buffer, then the input stream position range for this unit may be calculated as follows: $\begin{aligned} \text{start_pos} &= \text{CUR_INPUT_STREAM_POS} + \text{offset} \\ \text{end_pos} &= \text{CUR_INPUT_STREAM_POS} + \text{offset} + \text{size} \end{aligned}$
Actual Parameters	<i>p_xa_module_obj</i> <i>api_obj</i> – Pointer to API structure. <i>i_cmd</i> XA_API_CMD_GET_CONFIG_PARAM <i>i_idx</i> XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS <i>pv_value</i> &ui_cur_input_stream_pos – Pointer to the current input stream position variable.
Restrictions	The current input stream position counter is 32 bits and will, therefore, overflow and wrap around if the input stream length is more than $2^{32}-1$ bytes. This command is available in API version 1.15 or later.

Example

```
unsigned int ui_cur_input_stream_pos;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_CONFIG_PARAM,
    XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS,
    (void *) &ui_cur_input_stream_pos);
```

Errors

- Common API Errors.

Table 2-36 XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS
Description	This command reads the input stream position of the unit (for example, frame) corresponding to the generated (decoded or encoded) output data block. That is, if the main processing (DO_EXECUTE) call into the library generates any data in the output buffer, then this command reads the total number of input bytes consumed until the start of the unit that has been processed and placed into the output buffer. For example, if the application layer places a unit in the input buffer at input stream position <code>start_pos</code> (see Table 2-35), when the library generates the decoded or encoded data corresponding to this unit, it sets GEN_INPUT_STREAM_POS to <code>start_pos</code> .
Actual Parameters	<i>p_xa_module_obj</i> <i>api_obj</i> – Pointer to API structure. <i>i_cmd</i> XA_API_CMD_GET_CONFIG_PARAM <i>i_idx</i> XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS <i>pv_value</i> &<i>ui_gen_input_stream_pos</i> – Pointer to the input stream position of the generated data variable.
Restrictions	The input stream position of the generated data counter is 32 bits and will, therefore overflow and wrap-around, if the input stream length is more than $2^{32}-1$ bytes. This command is available in API version 1.15 or later.

Example

```

unsigned int ui_gen_input_stream_pos;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS,
                  (void *) &ui_gen_input_stream_pos);

```

Errors

- Common API Errors.

2.6.19 XA_API_CMD_SET_CONFIG_PARAM

Table 2-37 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS
Description	This command resets the current input stream position. For more information, see Table 2-35.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS pv_value &ui_cur_input_stream_pos – Pointer to the current input stream position variable.
Restrictions	This command is available in API version 1.15 or later.

Example

```
unsigned int ui_cur_input_stream_pos = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS,
                  (void *) &ui_cur_input_stream_pos);
```

Errors

- Common API Errors.

3. HiFi DSP AAC Decoder

The HiFi DSP AAC Decoder conforms to the generic codec API. The following flowchart for the AAC decoder integration details the sequence of commands used on the example testbench.

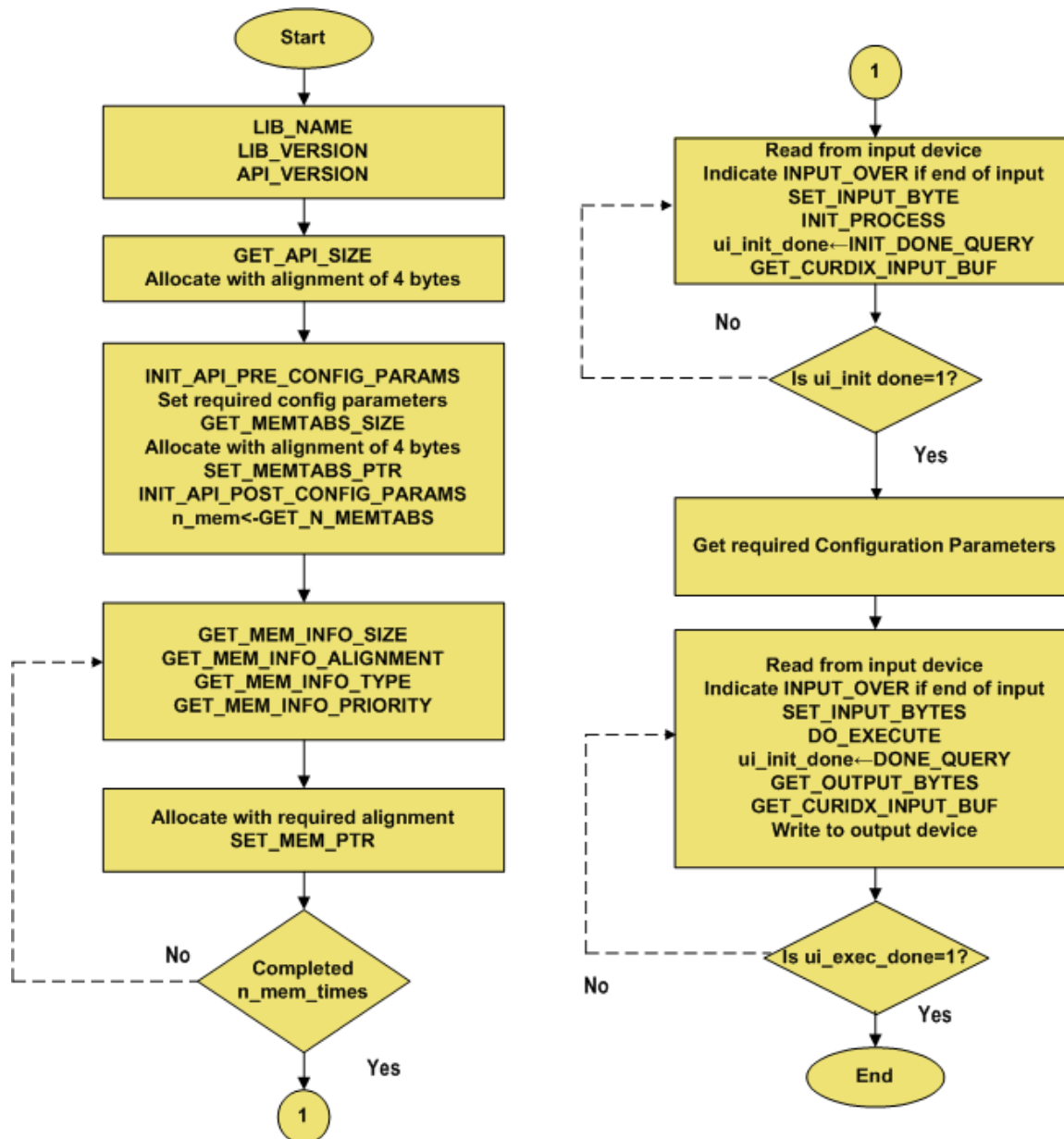


Figure 3 Flow Chart for AAC Decoder Integration

3.1 AAC Decoder-Specific Files

Each AAC Decoder package contains an API header file (.h) and one of the seven libraries (.a). The name and description of each file are listed below. In this document, each file may be referred to with a shorter name, as mentioned in the table.

- AAC Decoder API header file (in the include/aac_dec directory):
 - xa_aac_dec_api.h
- AAC Decoder Libraries:

Table 3-1 Library Files (in the lib directory):

Library Name	Description	Referred to as:
xa_aac_dec.a	stereo plain AAC	aac
xa_aacplus_dec.a	stereo aacPlus v1	aacplus
xa_aacplus_v2_dec.a	stereo aacPlus v2	aacplus_v2
xa_aacmch_dec.a	multi-channel plain AAC	aacmch
xa_aacmchplus_dec.a	multi-channel aacPlus v1	aacmchplus
xa_aacmchplus_v2_dec.a	multi-channel aacPlus v2	aacmchplus_v2
xa_aac_loas_dec.a	stereo plain AAC with LOAS/LATM support	aac_loas
xa_aacplus_loas_dec.a	stereo aacPlus v1 with LOAS/LATM support	aacplus_loas
xa_aacplus_v2_loas_dec.a	stereo aacPlus v2 with LOAS/LATM support	aacplus_v2_loas
xa_aacmch_loas_dec.a	multi-channel plain AAC with LOAS/LATM support	aacmch_loas
xa_aacmchplus_loas_dec.a	multi-channel aacPlus v1 with LOAS/LATM support	aacmchplus_loas
xa_aacmchplus_v2_loas_dec.a	multi-channel aacPlus v2 with LOAS/LATM support and DRC Processing	aacmchplus_v2_loas

Note aacmch, aacmchplus, aacmchplus_v2, aacmch_loas, aacmchplus_loas, and aacmchplus_v2_loas libraries are collectively called “Multi-Channel libraries”.

Note aac, aacplus, aacplus_v2, aac_loas, aacplus_loas, and aacplus_v2_loas libraries are grouped as “Stereo libraries”.

- The AAC decoder API call is defined as:

```
XA_ERRORCODE xa_aac_dec(xa_codec_handle_t p_xa_module_obj,
                        WORD32             i_cmd,
                        WORD32             i_idx,
                        pVOID              pv_value);
```

3.2 AAC Decoder-Specific Error Codes

Besides standard error codes explained in Section 2, AAC decoder APIs may also report error codes specific to AAC decoder libraries. These errors are classified into three classes:

- API Errors.
- Configuration Errors.
- Process Errors.

To simplify the text, the following terminologies are used in this section:

- INIT API or INIT process:
Calling the decoder API for XA_API_CMD_INIT command with subcommand XA_CMD_TYPE_INIT_PROCESS.
- EXEC API or EXEC process:
Calling the decoder API for XA_API_CMD_EXECUTE command with subcommand XA_CMD_TYPE_DO_EXECUTE.
- Config API:
Calling the decoder API for XA_API_CMD_SET_CONFIG_PARAM or XA_API_CMD_GET_CONFIG_PARAM with any subcommand.
- Transport layer:
The layer of the decoder that parses the transport header (ADTS, LOAS/LATM) or storage header (ADIF) and extracts the raw_data_block() (Refer to subpart 4 of [2]) from the input data.
- Raw Decoder:
The decoder layer parses raw_data_block() and decodes it, producing PCM output.

3.2.1 API Errors

API Errors are errors reported by the decoder when the application tries to call an API command or subcommand that is not supposed to be called.

For example:

- INIT/EXEC API cannot be called before allocating required memories.
- EXEC API cannot be called before a successful call of INIT API.
- Specific Config parameters can be obtained only after successful INIT.
- Specific Config parameters may not be supported for the bitstream format under decoding (e.g., MPEGID is not present in LOAS/LATM streams).

The API errors specific to AAC decoder libraries are as follows:

- `XA_AACDEC_API_NONFATAL_INVALID_API_SEQ`

Description: This error is reported in case of an invalid API sequence or unexpected API call. The decoder ignores the call.

Required or suggested actions: The application should not use the return parameters. The application programmer may also consider modifying the code to avoid this error. Refer to Figure 3 for the correct API sequence and Section 3.3 for accepted config API calls.

- `XA_AACDEC_API_NONFATAL_CMD_TYPE_NOT_SUPPORTED`

Description: This error is reported when a specific config API is not supported for the bitstream format under decoding.

Note: This error is an API error. It is only reported by config APIs.

Required or suggested actions: The application shall not use the returned parameters. The application programmer may consider modifying the code to avoid this non-fatal error.

- `XA_AACDEC_API_FATAL_INVALID_API_SEQ`

Description: This error is reported in case of an invalid API sequence or unexpected API call, and the decoder cannot proceed further.

Required or suggested actions: The application code shall be modified. Refer to Figure 3 for the correct API sequence and Section 3.3 for accepted config API calls.

3.2.2 Configuration Errors

Configuration errors are reported when a configuration subcommand fails. The failure may be due to an invalid config parameter value provided by the application, or the config parameter queried is not yet read from the stream. These errors can also be due to incorrect usage of config APIs for the given stream format. Config APIs may also return the common errors described in Section 2.6.1 and API errors described in Section 3.2.1.

The following are the common errors reported by configuration subcommands:

- `XA_AACDEC_CONFIG_NONFATAL_PARAMS_NOT_SET`

Description: This error is reported when a specific parameter is not yet read from a field in the encoded stream.

Required or suggested actions: The application must not use the returned parameter.

- `XA_AACDEC_CONFIG_FATAL_INVALID_EXTERNALBITRATE`

Description: This error is reported when an invalid bitrate is read from the command line arguments.

Required or suggested actions: Verify and provide the correct bitrate to the application.

- `XA_AACDEC_CONFIG_NONFATAL_CPID_NOT_PRESENT`

Description: This non-fatal error is reported when no copyright ID exists in the bitstream.

Required or suggested actions: Use a valid encoded bitstream with copyright ID embedded.

- XA_AACDEC_CONFIG_NONFATAL_PARTIAL_CHANROUTING

Description: This non-fatal error is reported when the channel route is user-modified and needs an update, or a few channels remain unmapped to the output.

Required or suggested actions: Use the complete and updated channel route map.

- XA_AACDEC_CONFIG_NONFATAL_INVALID_GEN_STRM_POS

Description: This error is reported to indicate if the input stream position is stale or un-updated.

Required or suggested action: Update the input stream position correctly.

Configuration errors unique to specific configuration subcommands are explained in sections 3.3.1 and 3.3.2, along with the configuration subcommands.

3.2.3 Process Errors

Process errors occur during the initialization or processing of API calls. Typically, these errors are caused by (but not limited to) the following:

- Invalid or missing configuration parameters.
- Wrong input and output buffer settings.
- Stream parsing errors.

The following process errors are specific to the AAC decoder:

- XA_AACDEC_EXECUTE_NONFATAL_INSUFFICIENT_FRAME_DATA

Description: The input buffer has insufficient data for initialization or execution.

Required or suggested actions:

The application needs to feed more data into the input buffer.

- XA_AACDEC_EXECUTE_NONFATAL_RUNTIME_INIT_RAMP_DOWN

Description: This non-fatal status code may be returned after a DO_EXECUTE call following a RUNTIME_INIT command for SBR streams. The output ramp down of SBR decoding extends to two frames, and this status code indicates that the ramp down has not finished yet.

Required or suggested actions: This error is for information only, and no explicit action is required from the application.

- XA_AACDEC_EXECUTE_NONFATAL_RAW_FRAME_PARSE_ERROR
XA_AACDEC_EXECUTE_FATAL_RAW_FRAME_PARSE_ERROR

Description: The decoder reports these errors when it encounters errors while parsing a raw frame. Both of these two errors can occur during the INIT or EXEC process. They should be addressed differently.

Required or suggested actions:

- Case 1: FATAL error during INIT. The decoder cannot initialize the raw decoder due to invalid sampling frequency or memory corruption (detected as NULL pointers). The application should revalidate the sampling frequency and other memory allocations and retry.
 - Case 2: FATAL error during EXEC. The application should stop the decoding process and feed another stream.
 - Case 3: NONFATAL error during INIT. The application can discard `bytes_consumed` or a single byte and continue the INIT process.
 - Case 4: NONFATAL error during EXEC. The application can continue decoding without any action. The current frame data will be discarded.
 - In the case of ADIF/ RAW transport formats, the application must feed the data from the next frame start. In this case, it is assumed that the application knows about the frame boundaries.
Note: In the EXEC process of ADTS, LOAS, or LATM streams, the decoder reports only NONFATAL errors.
- **XA_AACDEC_EXECUTE_NONFATAL_STREAM_CHANGE**
Description: In the case of ADTS/LOAS/LATM streams, this error may be reported if the decoder detects a change in one or more stream parameters.
Required or suggested actions: If the application confirms this as a stream change, it should stop the EXEC process and proceed with the INIT process so that the decoder starts decoding the new stream. If the application believes this error may be due to a stream error, it can discard one single byte and continue decoding. The current frame in which the parameter change is detected will be discarded.
 - **XA_AACDEC_EXECUTE_NONFATAL_HEADER_NOT_FOUND**
Description: This error is returned in the case of ADTS/LATM streams when no syncword is found in the data available in the input buffer or there are bytes to discard before the detected syncword.
Required or suggested actions: The application can continue the normal process.¹
 - **XA_AACDEC_EXECUTE_NONFATAL_UNSUPPORTED_FEATURE**
XA_AACDEC_EXECUTE_FATAL_UNSUPPORTED_FEATURE
Description: The decoder may report one of the `UNSUPPORTED_FEATURE` errors if it detects an unsupported feature in the stream. The FATAL error is reported for streams of type ADIF, or raw NONFATAL error is reported for streams of type ADTS, LOAS, or LATM.
Required or suggested actions: If the application confirms that the stream contains raw features not supported by the decoder, it should stop decoding. If the application thinks this error may be reported due to stream corruption.
 - In the case of ADTS, LOAS, or LATM, since the decoder's transport layer handles the bytes consumed, the application can continue decoding without further action.
 - In the case of raw or ADIF, the application should discard the current frame and feed the decoder from the next frame. In this case, it is assumed that the application knows about the frame boundary.
 - **XA_AACDEC_EXECUTE_NONFATAL_PARTIAL_LAST_FRAME**

¹ Be aware that the raw decoder is internally reset, as the decoder cannot confirm if any frame drop has caused this error.

Description: The decoder finds that the data in the input buffer is insufficient to decode a frame after receiving `input_over` from the application. The decoder consumes all the bytes in the buffer before it returns. It also sets `exec_done` in this scenario.

Required or suggested actions: The application should handle the `exec_done` message from the decoder.

■ `XA_AACDEC_EXECUTE_NONFATAL_HEADER_ERROR`

Description: The decoder reports this error when it detects an error while parsing the transport header.

Required or suggested actions: Although this is a non-fatal error, the application may consider this a fatal error in case of LATM/ADIF decoding and start feeding data from the next frame. In the case of ADTS/LOAS, no special action is required to form the application. Resync occurs in the decoder in the next EXEC / INIT API call.

■ `XA_AACDEC_EXECUTE_NONFATAL_ROUTING_ABSENT_CH_IGNORED`

Description: The decoder reports this error when it detects extra channels while validating the channel routing set by the application. The example scenario is given below.

- The input stream has two channels, but channel routing is done for X channels (with `-nX` option, where X is > 2). For example, `-n4 -0R -1L -2l -3r`. Here, `-2l`, `-3r` are ignored.

Required or suggested actions: The application should ignore this error as it is meant only for information.

■ `XA_AACDEC_EXECUTE_NONFATAL_EMPTY_INPUT_BUFFER`
`XA_AACDEC_EXECUTE_FATAL_EMPTY_INPUT_BUFFER`

Description: This error is reported when the decoder is called without any input data (specifically, transition in input state) and `input_over` is set. In the case of the EXEC process, a NONFATAL error indicates the end of decoding. In the case of INIT, a FATAL error indicates unsuccessful initialization.

Required or suggested actions: The application should handle the error and initialize the decoder with another stream to continue decoding.

■ `XA_AACDEC_EXECUTE_FATAL_ERROR_IN_CHANROUTING`

Description: This error is reported when there is a conflict between the channel routing configured from the application and the number of channels present in the stream under decoding.

Required or suggested actions: The application should reconfigure the channel routing specification and call the decoder again.

■ `XA_AACDEC_EXECUTE_FATAL_UNKNOWN_STREAM_FORMAT`

Description: This error is reported when the decoder cannot understand the stream format.

Required or suggested actions: The application may use `SET_CONFIG_PARAM` with the `EXTERNALBSFORMAT` subcommand to specify the bitstream format.

- For raw and LATM streams, the application should provide the stream format.
- For ADTS and LOAS streams, the auto-detection logic might have failed due to stream error, and the application may set the stream format. If the failure continues, the stream may be corrupted.

■ `XA_AACDEC_EXECUTE_FATAL_ADIF_HEADER_NOT_FOUND`

Description: The decoder could not find the ADIF header.

Required or suggested actions: The application should handle the error and initialize the decoder with another stream to continue decoding.

- XA_AACDEC_EXECUTE_NONFATAL_NEXT_SYNC_NOT_FOUND

Description: The decoder could not find the header for the next frame.

Required or suggested actions: The application should handle the error and initialize the decoder with another stream to continue decoding.

3.3 Configuration Parameters

The application can configure the AAC decoder using the “SET CONFIG” API. The application can read parameters specific to a decoded stream and the current context of the decoding process using the GET CONFIG API. These configuration parameters are explained in detail in sections 3.3.1 and 3.3.2.

List of parameters that can be configured by SET_CONFIG:

Configuration parameters supported by all AAC decoder libraries:

- external_sampling_rate (refer to Table 3-2)
- external_bsformat (refer to Table 3-3)
- mono_to_stereo (refer to Table 3-4)
- pcm_wdsz2 (refer to Table 3-5)
- outnchans2 (refer to Table 3-6)
- chanrouting2 (refer to Table 3-7)
- zero_unused_chans (refer to Table 3-8)
- input_bitoffset (refer to Table 3-9)
- enableframeByFrameDecode (refer to Table 3-18)
- bmpeg4_amd4 (refer to Table 3-19)

Configuration parameters supported by libraries that can decode HE-AAC streams:

- bdownsample (refer to Table 3-10)
- downmix (refer to Table 3-11)
- sbr_signaling (refer to Table 3-12)

Configuration parameters supported only by the aacmchplus_v2_loas library:

- apply_prl (refer to Table 3-13)
- target_level (refer to Table 3-14)
- apply_drc (refer to Table 3-15)
- drc_compress (refer to Table 3-16)
- drc_boost (refer to Table 3-17)

List of parameters that can be queried by GET_CONFIG:

The application can query for parameters related to the input stream / decoding process. Except for those mentioned below, these parameters can be queried only after a successful INIT.

² The pcm_wdsz, outnchans, and chanrouting parameters must be set before the postconfig INITAPI call.

Parameters that can be queried from any library:

- output_sample_rate (refer to Table 3-20)
- num_channels (refer to Table 3-21)
- pcm_wdsz3 (refer to Table 3-22)
- sbr_type (refer to Table 3-23)
- aac_samplerate (refer to Table 3-24)
- data_rate4 (refer to Table 3-25)
- chanmap (refer to Table 3-26)
- acmod (refer to Table 3-27)
- aac_format (refer to Table 3-28)
- out_num_channels (refer to Table 3-29)
- drc_ext_present (refer to Table 3-30)
- mpeg_id (refer to Table 3-31)
- is_original (refer to Table 3-32)
- copyright_id (refer to Table 3-33)
- enableframeByFrameDecode (refer to Table 3-39)
- bmpeg4_amd4 (refer to Table 3-40)
- ch_cfg_info (refer to Table 3-41)

Parameters that can be queried from Multi-Channel libraries:

- pce_status (refer to Table 3-34)
- downmix_metadata (refer to Table 3-35)
- downmix_level_dvb (refer to Table 3-36)
- parsed_drc_info (refer to Table 3-37)

Parameters that can be queried from the aacmchplus_v2_loas library only:

- prog_ref_level (refer to Table 3-38)

³ This parameter can be queried any time after preconfig init.

⁴ This parameter can be queried only after one successful frame decoding.

3.3.1 XA_API_CMD_SET_CONFIG_PARAM

Table 3-2 XA_AACDEC_CONFIG_PARAM_EXTERNALSAMPLINGRATE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_EXTERNALSAMPLINGRATE												
Description	This command sets the sampling rate of the basic AAC stream in the case of raw bitstreams.												
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_EXTERNALSAMPLINGRATE pv_value &externalsr – Pointer to the external sample rate Variable.												
Restrictions	■ This subcommand is required only for raw stream decoding. For all other formats, the value is “ignored”. ■ Note that for raw SBR streams, the basic AAC content sampling rate may be half the output sampling rate. ■ Valid values: <table><tr><td>8000</td><td>11025</td><td>12000</td></tr><tr><td>16000</td><td>22050</td><td>24000</td></tr><tr><td>32000</td><td>44100</td><td>48000</td></tr><tr><td>64000</td><td>88200</td><td>96000</td></tr></table> ■ The default value is 44100.	8000	11025	12000	16000	22050	24000	32000	44100	48000	64000	88200	96000
8000	11025	12000											
16000	22050	24000											
32000	44100	48000											
64000	88200	96000											

Example

```
int externalsr = 48000;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_EXTERNALSAMPLINGRATE,
(void *) &externalsr);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_EXTERNALSAMPLINGRATE occurs.

Table 3-3 XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT
Description	This command sets the bitstream format for the given stream (from the application). enum type XA_AACDEC_EBITSTREAM_TYPE is defined in xa_aac_dec_api.h
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT pv_value &bsformat – Pointer to the bitstream format variable. </pre>
Restrictions	■ The current version of the decoder supports only one of the following bitstream types. For all libraries: <pre> XA_AACDEC_EBITSTREAM_TYPE_AAC_RAW XA_AACDEC_EBITSTREAM_TYPE_AAC_ADTS XA_AACDEC_EBITSTREAM_TYPE_AAC_ADIF </pre> Libraries with LOAS also support <pre> XA_AACDEC_EBITSTREAM_TYPE_AAC_LOAS XA_AACDEC_EBITSTREAM_TYPE_AAC_LATM </pre> ■ If the external bitstream format is set, the auto-detection is disabled, and the library tries to decode the given stream with the specified format only. For example, suppose the bitstream format is set to ADTS, and an LOAS stream is provided. In that case, the decoding process will fail as the decoder interprets the LOAS stream as ADTS, and the generated output will be unpredictable. ■ If the external bitstream format is not set or is set to XA_AACDEC_EBITSTREAM_TYPE_AAC_UNKNOWN (default), the decoder activates the auto-detect functionality and tries to detect ADIF, ADTS, and LOAS streams. ■ The application must set this parameter for raw and LATM streams.

Example

```
XA_AACDEC_EBITSTREAM_TYPE  
bsformat=XA_AACDEC_EBITSTREAM_TYPE_AAC_RAW  
res = (*api_func)(api_obj,  
XA_API_CMD_SET_CONFIG_PARAM,  
XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT,  
(void *) &bsformat);
```

Errors

- When the input value is not valid,
XA_AACDEC_CONFIG_FATAL_INVALID_EXTERNALBSFORMAT occurs.

Table 3-4 XA_AACDEC_CONFIG_PARAM_TO_STEREO Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_TO_STEREO
Description	This command enables or disables the conversion of mono to stereo in the output buffer. If enabled, the mono signal is “replicated” in two (stereo) output channels.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_TO_STEREO pv_value &to_stereo – Pointer to the stereo conversion flag variable.
Restrictions	- Valid values: 0 - mono streams are presented as single-channel. 1 - mono streams are presented as two identical channels (default). - For Non-mono streams, this setting is ignored.

Example

```
int mono_to_stereo = 0;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_TO_STEREO,
(void *) &mono_to_stereo);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_TO_STEREO occurs.

Table 3-5 XA_AACDEC_CONFIG_PARAM_PCM_WDSZ Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_PCM_WDSZ
Description	This command sets the output PCM sample bit-width to 16 or 24 bits.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_PCM_WDSZ pv_value &pcm_wdsz – Pointer to the width of the PCM sample variable.
Restrictions	■ In 24-bit format, the samples are stored in the 24 MSBs of each output 32-bit word; the 8 LSBs are set to 0. ■ In 16-bit format, the samples are stored in 16-bit words. ■ Valid values: 16 - 16-bit PCM samples 24 - 24-bit PCM samples (default)

Example

```
int pcm_wdsz = 16;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_PCM_WDSZ,
(void *) &pcm_wdsz);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_PCM_WDSZ occurs.

Table 3-6 XA_AACDEC_CONFIG_PARAM_OUTNCHANS Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_OUTNCHANS
Description	This command sets the maximum number of channels supported by the decoder. Note that this configuration parameter also affects persistent, scratch, and output memory allocation. If a channel is not present in the encoded input stream, the corresponding sample value is set to zero in the output buffer.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_OUTNCHANS pv_value &outnchans – Pointer to the number of output channels variable.
Restrictions	Valid values: Stereo libraries: the default and only valid value is 2. Multi-Channel libraries: 2 through 8 (default 8)

Example

```
int outnchans = 6;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_OUTNCHANS,
(void *)&outnchans);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_OUTNCHANS occurs.

Table 3-7 XA_AACDEC_CONFIG_PARAM_CHANROUTING Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_CHANROUTING																
Description	Controls the output channel routing in the output PCM buffer. To set this parameter, the channel index is specified in byte 0 (bits 0-7), and the sample offset in the interleaving order is specified in byte 1 (bits 8 – 15) of the int variable chanrouting. Channel indices are as below: <table><tr><td>0</td><td>L</td><td>4</td><td>r</td></tr><tr><td>1</td><td>C</td><td>5</td><td>Sbl (or Cs)</td></tr><tr><td>2</td><td>R</td><td>6</td><td>Sbr</td></tr><tr><td>3</td><td>I</td><td>7</td><td>LFE</td></tr></table>	0	L	4	r	1	C	5	Sbl (or Cs)	2	R	6	Sbr	3	I	7	LFE
0	L	4	r														
1	C	5	Sbl (or Cs)														
2	R	6	Sbr														
3	I	7	LFE														
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_CHANROUTING pv_value &chanrouting – Pointer to the channel routing variable.																
Restrictions	■ To set up correctly, this parameter must be set outnchans times to specify the routing of all the output channels completely. An incomplete setup may lead to the failure of post-config initialization or during the decoder initialization process. Refer to Section 3.4 for supported channel routing. ■ The sample offset value should be less than the outnchans value. This setup is validated during post-configuration and during the raw decoder initializing process. ■ If this parameter is not set, the samples are routed as per their natural order (as they appear in the bit-stream) in the output buffer. To know the natural order or the order currently used, the application can query for chanmap (refer to Table 3-26).																

Example

```
unsigned int chanrouting;
/* Route the L channel to sample offset 0 in the output */
chanrouting = (0 << 8) | 0; /* L → 0 */
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_CHANROUTING,
(void *) &chanrouting);
/* Route the R channel to sample offset 1 in the output */
chanrouting = (1 << 8) | 2; /* R → 2 */
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_CHANROUTING,
(void *) &chanrouting);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_CHANROUTING occurs.

Table 3-8 XA_AACDEC_CONFIG_PARAM_ZERO_UNUSED_CHANS Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ZERO_UNUSED_CHANS
Description	Enable (1) or disable (0) zeroing of unused output channels.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ZERO_UNUSED_CHANS pv_value &zero_unused_chans – Pointer to the zero-unused-channels flag variable.
Restrictions	■ Valid values: ■ 0 – Output PCM samples corresponding to unused channels are left unmodified (Default). ■ 1 – enable zeroing of unused output channels. ■ Enabling this may lead to higher CPU load and memory bandwidth requirements in cases when the number of decoded channels (num_channels) is less than the maximum number of channels to be placed in the input buffer (outnchans).

Example

```
int zero_unused_chans = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_ZERO_UNUSED_CHANS,
(void *) &zero_unused_chans);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_ZERO_UNUSED_CHANS occurs.

Table 3-9 XA_AACDEC_CONFIG_PARAM_INPUT_BITOFFSET Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_INPUT_BITOFFSET
Description	This API discards a few bits in the input bitstream and is applicable only for raw stream decoding. In general, the decoder reads data from the first byte. In some cases (such as LATM), the raw data may not start from the first bit of the byte. In this case, the application can call this API to discard the bits before beginning the decoding process.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_INPUT_BITOFFSET pv_value &bit_offset – bit offset value (usually between 0-7).</pre>
Restrictions	■ This parameter can be changed during run-time ■ Valid values: 0-7. This API can also be used to discard a greater number of bytes by providing the value in bits. i.e., setting this value to 32 will discard the first four bytes in the bitstream. However, this feature is not fully tested; hence it is recommended to use values between 0 and 7. ■ The value set through this API will be cleared after every INIT/EXEC API call. ■ The bit offset will be assumed as 0 if this API is not called. ■ The bit offset value set through this API will be ignored if the input stream format is not raw. ■ If the bit_offset value is negative, it will be ignored.

Example

```

int bit_offset = 3;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_INPUT_BITOFFSET,
(void *) &bit_offset);
```

Errors

- No specific error

Table 3-10 XA_AACDEC_CONFIG_PARAM_BDOWNSAMPLE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_BDOWNSAMPLE
Description	This command sets the SBR downsample flag.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_BDOWNSAMPLE pv_value &bdownsample – Pointer to the SBR downsample flag variable.
Restrictions	■ Valid values: ■ 0 – disabled (default) ■ 1 – enabled ■ This setting will be ignored for non-SBR streams. ■ This subcommand is not available for aac, aac_loas, aacmch, and aacmch_loas libraries.

Example

```
int bdownsample = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_BDOWNSAMPLE,
(void *) &bdownsample);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_BDOWNSAMPLE occurs.

Table 3-11 XA_AACDEC_CONFIG_PARAM_BBITSTREAMDOWNMIX Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_BBITSTREAMDOWNMIX
Description	This command sets the downmix flag. The flag is used to configure the SBR tool to perform SBR processing on a downmixed mono channel instead of a channel pair. The mono channel is derived by downmixing the input channel pair.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_BBITSTREAMDOWNMIX pv_value &downmix – Pointer to the downmix flag variable.
Restrictions	■ Valid values: ■ 0 – disabled (default) ■ 1 – enabled ■ Enabling this flag applies SBR processing on the mono channel to reduce the computational requirement of the processing. If the input to SBR processing is a channel pair, the channels are downmixed into a mono channel and passed through SBR processing. ■ This setting will be ignored for non-SBR streams. ■ This setting will be ignored for non-parametric stereo streams. ■ This subcommand is unavailable for aac, aac_loas, aacmch, and aacmch_loas libraries.

Example

```
int downmix = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_BBITSTREAMDOWNMIX,
(void *) &downmix);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_BBITSTREAMDOWNMIX occurs.

Table 3-12 XA_AACDEC_CONFIG_PARAM_SBR_SIGNALING Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_SBR_SIGNALING
Description	This command enables/disables the SBR processing or sets it to auto mode so that the SBR tool can decode the input streams.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_SBR_SIGNALING pv_value &sbr_signaling – Pointer to the flag.
Restrictions	■ Valid values: ■ 0 – Disables the SBR tool during decoding of all types of input bitstreams. ■ 1 – Enables and applies the SBR tool during the decoding of all types of input bitstreams. This forces SBR processing even for plain AAC input streams. ■ 2 – Applies the SBR tool only for decoding of input bitstreams containing SBR information; Auto mode (default). ■ The sbr_signaling parameter controls and modifies the audio object type (sbr_type) returned by the library (Refer to Table 3-23). If sbr_signaling is 0, then sbr_type is always 0 (plain AAC audio object type). If sbr_signaling is 1, then sbr_type is either 1 or 2 (indicating aacPlus object decoding). ■ This subcommand is unavailable for aac, aac_loas, aacmch, and aacmch_loas libraries.

Example

```
int sbr_signaling = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_SBR_SIGNALING,
(void *)&sbr_signaling);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_FATAL_INVALID_SBR_SIGNALING occurs.

Table 3-13 XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_PRL Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_PRL
Description	Enable or disable the application of program reference level scaling to a desired target value specified through XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL (Refer to Table 3-14).
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_PRL pv_value &apply_prl – Pointer to a binary flag variable controlling the application of program ref level scaling.
Restrictions	■ This configuration is available only with the aacmchplus_v2_loas library. ■ This parameter can be changed during runtime. ■ Valid values: ■ 0 – No scaling is applied to the decoder output (default). ■ 1 – Dynamic range scaling is applied in the spectral domain.

Example

```
int apply_prl = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_APPLY_PRL,
(void *) &apply_prl);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_NONFATAL_INVALID_PRL_PARAMS occurs.

Table 3-14 XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL
Description	This is a 7-bit value to set the desired target level of the decoder output. This is only applicable if the program reference level is available in the bit stream. If the program reference level information is unavailable in the stream, then the output does not undergo scaling. In addition, the output of the decoder is scaled to the desired target output level only if the <code>apply_prl</code> parameter mentioned in Table 3-13 is set to 1.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to the API structure. <code>i_cmd</code> <code>XA_API_CMD_SET_CONFIG_PARAM</code> <code>i_idx</code> <code>XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL</code> <code>pv_value</code> <code>&target_level</code> – Pointer to a 7-bit value indicating desired output level.
Restrictions	■ This configuration is available only with <code>aacmchplus_v2_loas</code> Library. ■ This parameter can be changed during runtime. ■ This is a 7-bit unsigned value stored in a 32-bit word. The corresponding desired output level can be decided based on the following value: $\text{desired_target_level_in_dB} = -0.25 * \text{target_level}$. ■ Valid values: 0 to 127 (default: 124).

Example

```
float desired_output_level_db = -24.0;
int target_level = (int)(-4* desired_output_level_db);
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_TARGET_LEVEL,
(void *) &target_level);
```

Errors

- When the input value is not valid, `XA_AACDEC_CONFIG_NONFATAL_INVALID_PRL_PARAMS` occurs.

Table 3-15 XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_DRC Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_DRC
Description	Enable (1) or disable (0) application of dynamic range compression. The percentage of dynamic scaling is controlled by the drc_compress and drc_boost values described in Table 3-16 and Table 3-17.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ENABLE_APPLY_DRC pv_value &apply_drc – Pointer to a binary flag variable controlling the application of dynamic range compression.
Restrictions	■ This configuration is available only with aacmchplus_v2_loas library. ■ This parameter can be changed during runtime. ■ Valid values: ■ 0 – No dynamic range scaling is applied to the decoder output (Default) ■ 1 – Dynamic range scaling is applied in the spectral domain

Example

```
int apply_drc = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_APPLY_DRC,
(void *) &apply_drc);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_NONFATAL_INVALID_DRC_PARAMS occurs.

Table 3-16 XA_AACDEC_CONFIG_PARAM_DRC_COMPRESS_FAC Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DRC_COMPRESS_FAC
Description	This parameter controls the scaling of loud signals so that they are compressed, as mentioned in the MPEG-4 Audio standard ^[2] . This parameter is passed as a fraction stored in a 9.23 format. The scaling is applied only if the <code>apply_drc</code> flag described in Table 3-15 is set.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to the API structure. <code>i_cmd</code> XA_API_CMD_SET_CONFIG_PARAM <code>i_idx</code> XA_AACDEC_CONFIG_PARAM_DRC_COMPRESS_FAC <code>pv_value</code> <code>&drc_compress</code> – Pointer to a fractional variable controlling the percentage of the application of dynamic range compression (cut).
Restrictions	■ This configuration is available only with the <code>aacmchplus_v2_loas</code> library. ■ This parameter can be changed during runtime. ■ These are integers representing values in a 9.23 format. For example, dynamic range compression (cut) of 100% is represented as a 32-bit integer value of 0x00800000. Dynamic range compression of 50% shall be represented by 0x00400000.

Example

```
float percent_drc_cut = 60.0;
int drc_compress=(int) (percent_drc_cut/100.0*(1<<23));
res = (*api_func) (api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DRC_COMPRESS_FAC,
(void *) &drc_compress);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_NONFATAL_INVALID_DRC_PARAMS occurs.

Table 3-17 XA_AACDEC_CONFIG_PARAM_DRC_BOOST_FAC Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DRC_BOOST_FAC
Description	This parameter controls the scaling of weak signals so that they are boosted, as mentioned in the MPEG-4 Audio standard [2]. This parameter is passed as a fraction in a 9.23 format. The scaling is applied only if the <code>apply_drc</code> flag described in Table 3-15 is set.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to the API structure. <code>i_cmd</code> <code>XA_API_CMD_SET_CONFIG_PARAM</code> <code>i_idx</code> <code>XA_AACDEC_CONFIG_PARAM_DRC_BOOST_FAC</code> <code>pv_value</code> <code>&drc_boost</code> – Pointer to a fractional variable controlling the percentage of the application of dynamic range compression (boost).
Restrictions	■ This configuration is available only with the <code>aacmchplus_v2_loas</code> library. ■ This parameter can be changed during runtime. ■ These are integers representing values in a 9.23 format. For example, a dynamic range boost of 100% is represented as a 32-bit integer value of <code>0x00800000</code>. A dynamic range boost of 25 % shall be represented by <code>0x00200000</code>.

Example

```
float percent_drc_boost = 40.0;
int drc_boost = (int) (percent_drc_boost/100.0*(1<<23));
res = (*api_func) (api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DRC_BOOST_FAC,
(void *) &drc_boost);
```

Errors

- When the input value is not valid, `XA_AACDEC_CONFIG_NONFATAL_INVALID_DRC_PARAMS` occurs.

Table 3-18 XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE
Description	The command sets the decoder's operating mode: streaming mode or frame-by-frame mode for ADTS/LOAS streams. In streaming mode, the decoder is designed to confirm the frame's validity by looking ahead and verifying the next frame header. This mechanism allows the decoder to check and notify the application about stream change by connecting the frame info between current, previous, and future frame headers. In frame-by-frame mode, the decoder will not look ahead for the next frame header to check the frame validity, and stream change detection is disabled. The frame-by-frame mode is less robust in error-prone systems. The application must detect events such as bitstream errors and stream changes and handle the decoder properly when such events occur.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE pv_value &enableframeByFrameDecode – Pointer to the operating mode variable. </pre>
Restrictions	■ This parameter can be changed during runtime. ■ Enabling this mode will disable stream change detection. ■ This setting will not affect the behavior of auto format detection during init, and a look ahead is still required to detect the stream. You must set the stream format to use frame-by-frame decode mode correctly. ■ Valid value: 0 (streaming mode, the default), 1 (frame-by-frame mode).

Example

```
int enableframeByFrameDecode = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE
(void *) &enableframeByFrameDecode);
```

Errors

- No specific error.

Table 3-19 XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE
Description	This command enables/disables the MPEG4 amendment4 flag.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE pv_value &bmpeg4_amd4 – Pointer to the MPEG4 amendment flag variable.
Restrictions	■ Valid values: ■ 0 – disabled ■ 1 – enabled (default)

Example

```
int bmpeg4_amd4 = 1;
res = (*api_func)(api_obj,
XA_API_CMD_SET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE,
(void *) & bmpeg4_amd4);
```

Errors

- When the input value is not valid, XA_AACDEC_CONFIG_NONFATAL_INVALID_PARAM_VALUE occurs.

3.3.2 XA_API_CMD_GET_CONFIG_PARAM

Table 3-20 XA_AACDEC_CONFIG_PARAM_OUT_SAMPLERATE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_OUT_SAMPLERATE
Description	This command gets the output sample rate (in Hz).
Actual Parameters	<pre>p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_OUT_SAMPLERATE pv_value &samp_freq – Pointer to the output sample rate variable.</pre>
Restrictions	None

Example

```
int samp_freq;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_OUT_SAMPLERATE,
(void *) &samp_freq);
```

Errors

- No specific error.

Table 3-21 XA_AACDEC_CONFIG_PARAM_NUM_CHANNELS subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_NUM_CHANNELS
Description	This command gets the number of decoded channels present in the output buffer. Values in the range of 1 to 8.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_NUM_CHANNELS pv_value &num_channels – Pointer to the output number of channels variable.
Restrictions	None

Example

```
int num_channels;  
res = (*api_func)(api_obj,  
XA_API_CMD_GET_CONFIG_PARAM,  
XA_AACDEC_CONFIG_PARAM_NUM_CHANNELS,  
(void *) &num_channels);
```

Errors

- No specific error.

Table 3-22 XA_AACDEC_CONFIG_PARAM_PCM_WDSZ Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_PCM_WDSZ
Description	This command gets the output bit-width. Possible return values are 16 and 24. This returns a default value or the value set by the application using XA_API_CMD_SET_CONFIG_PARAM with the XA_AACDEC_CONFIG_PARAM_PCM_WDSZ subcommand as explained in Table 3-5.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_PCM_WDSZ pv_value &pcm_wdsz – Pointer to the width of the PCM sample variable. </pre>
Restrictions	This API is optional, and the application can also determine the information provided by this API.

Example

```

int pcm_wdsz;
res = (*api_func) (api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_PCM_WDSZ,
(void *) &pcm_wdsz);

```

Errors

- No specific error.

Table 3-23 XA_AACDEC_CONFIG_PARAM_SBR_TYPE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_SBR_TYPE
Description	This command gets the SBR type for the stream under decoding. 0 Plain AAC 1 AAC+ V1 (SBR) 2 AAC+ V2 (Parametric Stereo)
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_SBR_TYPE pv_value &sbr_type – Pointer to the SBR type variable.
Restrictions	None

Example

```
int sbr_type;  
res = (*api_func)(api_obj,  
XA_API_CMD_GET_CONFIG_PARAM,  
XA_AACDEC_CONFIG_PARAM_SBR_TYPE,  
(void *) &sbr_type);
```

Errors

- No specific error.

Table 3-24 XA_AACDEC_CONFIG_PARAM_AAC_SAMPLERATE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_AAC_SAMPLERATE
Description	This command gets the sample rate (in Hz) at which the plain AAC decoder operates. When SBR is used, this figure can be half the output sample rate.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_AAC_SAMPLERATE pv_value &aac_samplerate – Pointer to the AAC sample rate variable.
Restrictions	None

Example

```
int aac_samplerate;  
res = (*api_func)(api_obj,  
XA_API_CMD_GET_CONFIG_PARAM,  
XA_AACDEC_CONFIG_PARAM_AAC_SAMPLERATE,  
(void *) &aac_samplerate);
```

Errors

- No specific error.

Table 3-25 XA_AACDEC_CONFIG_PARAM_DATA_RATE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DATA_RATE
Description	This command gets the approximate average data rate (in bits) of the encoded stream. For ADIF streams, the data rate is read from the ADIF header, and the value remains the same throughout the decoding. For non-ADIF streams, the data rate is calculated and updated after every successful frame decoding.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_DATA_RATE pv_value &data_rate – Pointer to the input data rate variable. </pre>
Restrictions	The average bit rate stabilizes over time; hence, the application may start reading after a few (>15) data frames.

Example

```

int data_rate;
res = (*api_func) (api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DATA_RATE,
(void *) &data_rate);

```

Errors

- XA_AACDEC_CONFIG_NONFATAL_DATA_RATE_NOT_SET is returned if this API is called before the first successful frame decoding.

Table 3-26 XA_AACDEC_CONFIG_PARAM_CHANMAP Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_CHANMAP
Description	This parameter specifies how the channels are arranged in the output buffer. The nibbles 0 to 7 of the chanmap variable are set to channel index values based on their sample offsets in the interleaved output PCM buffer. The Channel index to Channel mapping is provided in Table 3-7. If a channel (with channel index C) appears at sample offset N in the interleaved output PCM buffer, then the Nth nibble of the chanmap parameter is set to C. The unused sample offsets are set to a value of 0xF. For example: chanmap = 0xFFFF5201 indicates that: the center channel (channel index 1) is present at sample offset of 0, the left channel (channel index 0) is present at sample offset 1, the right channel (channel index 2) is present at sample offset 2, the center-surround (channel index 5) is present at sample offset 3. There are no other decoded channels present at the remaining sample offsets.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_CHANMAP pv_value &chanmap – Pointer to the chanmap variable. </pre>
Restrictions	None

Example

```

int chanmap;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_CHANMAP,
(void *) &chanmap);

```

Errors

- No specific error.

Table 3-27 XA_AACDEC_CONFIG_PARAM_ACMOD Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ACMOD
Description	Information about the input bitstream's audio coding mode (encoded channel configuration). enum type XA_AACDEC_CHANNELMODE is defined in xa_aac_dec_api.h.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ACMOD pv_value &acmod – Pointer to the audio coding mode variable.
Restrictions	None

Example

```
XA_AACDEC_CHANNELMODE acmod;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_ACMOD,
(void *) &acmod);
```

Errors

- No specific error.

Table 3-28 XA_AACDEC_CONFIG_PARAM_AAC_FORMAT Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_AAC_FORMAT
Description	This command gets the stream format of the input stream. If the stream format were set using XA_AACDEC_CONFIG_PARAM_EXTERNALBSFORMAT, the same would be returned. If the decoder detected the stream format automatically, the detected value would be returned. The enum type XA_AACDEC_EBITSTREAM_TYPE is defined in xa_aac_dec_api.h
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_AAC_FORMAT pv_value &aac_format – Pointer to the AAC format variable. </pre>
Restrictions	None

Example

```

XA_AACDEC_EBITSTREAM_TYPE aac_format;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_AAC_FORMAT,
(void *)&aac_format);

```

Errors

- No specific error.

Table 3-29 XA_AACDEC_CONFIG_PARAM_OUTNCHANS Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_OUTNCHANS
Description	This returns a default value or the value set by the application using XA_API_CMD_SET_CONFIG_PARAM with XA_AACDEC_CONFIG_PARAM_OUTNCHANS subcommand explained in Table 3-6.
Actual Parameters	<pre>p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_OUTNCHANS pv_value &outnchans – Number of output channels (both valid and invalid).</pre>
Restrictions	This API is optional, and the application can also determine the information provided by this API.

Example

```
int outnchans;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_OUTNCHANS,
(void *) &outnchans);
```

Errors

- No specific error.

Table 3-30 XA_AACDEC_CONFIG_PARAM_DRC_EXT_PRESENT Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DRC_EXT_PRESENT
Description	This parameter returns a flag to indicate whether the payload contains the DRC extension payload or not. Return value 1 indicates that the DRC extension is present, and 0 indicates that it is absent.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_DRC_EXT_PRESENT pv_value &drc_ext_flag – Pointer to the DRC present variable.
Restrictions	None

Example

```
int drc_ext_flag;  
res = (*api_func)(api_obj,  
XA_API_CMD_GET_CONFIG_PARAM,  
XA_AACDEC_CONFIG_PARAM_DRC_EXT_PRESENT,  
(void *) &drc_ext_flag);
```

Errors

- No specific error.

Table 3-31 XA_AACDEC_CONFIG_PARAM_MPEG_ID Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_MPEG_ID
Description	This parameter returns a flag to indicate whether the payload is MPEG2-AAC or MPEG4-AAC. The value is set to 1 if the audio data in the ADTS stream is MPEG-2 AAC (see ISO/IEC 13818-7) and 0 if it is MPEG-4.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_MPEG_ID pv_value &Mpeg_ID – Pointer to a variable Mpeg_ID. </pre>
Restrictions	This value is available only if the payload is in ADTS format (as explained in Section 1.A.4.3 of ISO/IEC 14496-3).

Example

```

int Mpeg_ID;
res = (*api_func) (api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_MPEG_ID,
(void *) &Mpeg_ID);

```

Errors

- No specific error.

Table 3-32 XA_AACDEC_CONFIG_PARAM_ORIGINAL_OR_COPY Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ORIGINAL_OR_COPY
Description	This parameter returns a flag to indicate whether the bitstream is original or copy. The value is set to 1 if the bitstream is original and 0 if the bitstream is copied.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ORIGINAL_OR_COPY pv_value &orig_ID – Pointer to the original/copy indication variable.
Restrictions	This value is available only if the bitstream is in ADTS or ADIF format (as explained in Section 2.4.2.3 of ISO/IEC 11172-3).

Example

```
int orig_ID;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_ORIGINAL_OR_COPY,
(void *) &orig_ID);
```

Errors

- No specific error.

Table 3-33 XA_AACDEC_CONFIG_PARAM_COPYRIGHT_ID_PTR Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_COPYRIGHT_ID_PTR
Description	This parameter returns a pointer to an array of nine unsigned characters (representing 72 bits of Copyright Identification information received from the bitstream).
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_COPYRIGHT_ID_PTR pv_value &copyright_info – Pointer to the copyright_info array.
Restrictions	■ This API exposes the pointer to an internal element of the decoder. The application should treat this pointer as READ-ONLY and read only 9 bytes from the pointer. ■ This information is available only if the bitstream is in ADTS or ADIF format (as explained in Section 1.A.3 of ISO/IEC 14496-3).

Example

```

unsigned char *copyright_info;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_COPYRIGHT_ID_PTR,
(void *) &copyright_info);

```

Errors

- No specific error.

Table 3-34 XA_AACDEC_CONFIG_PARAM_PCE_STATUS Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_PCE_STATUS
Description	This parameter returns a value to indicate whether a PCE (Program Config Element) is present in the encoded frame enum type <code>xa_aac_dec_pce_status</code> is defined in <code>xa_aac_dec_api.h</code> LOAS/LATM PCE status is read from LATMheader. If PCE is not present in the LATM header, it is read from the raw frame. ADTS, raw PCE status is read from the raw frame. ADIF PCE status read from ADIF header.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to the API structure. <code>i_cmd</code> <code>XA_API_CMD_GET_CONFIG_PARAM</code> <code>i_idx</code> <code>XA_AACDEC_CONFIG_PARAM_PCE_STATUS</code> <code>pv_value</code> <code>&PCEstatus</code> – Pointer to a variable PCE status as defined below.
Restrictions	This parameter is available only with Multi-Channel Libraries.

Example

```

xa_aac_dec_pce_status PCEstatus;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA,
(void *) &PCEstatus);

```

Errors

- No specific error.

Table 3-35 XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA
Description	This parameter is a pointer to a structure that is filled with data related to the downmix present in (PCE – program config element) as specified in the MPEG4-AAC audio document ISO/IEC 14496-3 in Section 4.4.1.1^[2]. This information is either a part of the LATMheader or a part of the syntax element of the core encoded frame. struct type <code>xa_aac_dec_dwnmix_metadata_t</code> is defined in <code>xa_aac_dec_api.h</code>.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA pv_value &pDmxMetaData – Pointer to a structure containing downmix data as defined below. </pre>
Restrictions	This parameter is available only with Multi-Channel Libraries.

Example

```

xa_aac_dec_dwnmix_metadata_t dmx_meta_data;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DWNMIX_METADATA,
(void *) &dmx_meta_data);

```

Errors

- No specific error.

Table 3-36 XA_AACDEC_CONFIG_PARAM_DWNMIX_LEVEL_DVB Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_DWNMIX_LEVEL_DVB
Description	This parameter is a pointer to a structure filled with data related to downmix levels present in ancillary data (DSE), as specified in ETSI TS101.154 v1.9.1. These are relevant for downmixing the outputs to stereo for DVB-like applications. struct type <code>xa_aac_dec_dwnmix_level_dvb_info_t</code> is defined in <code>xa_aac_dec_api.h</code>.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_DWNMIX_LEVEL_DVB pv_value &dvb_dmx_info – Pointer to the structure containing downmix level data as defined below. </pre>
Restrictions	■ This parameter is available only with Multi-Channel libraries. ■ The DVB info structure values retain their old values if the element <code>new_dvb_downmix_data</code> equals 0. If the parsed data element does not contain timecodes, then the <code>*_timecode_value</code> and <code>*_timecode_on</code> parameters remain 0.

Example

```

xa_aac_dec_dwnmix_level_dvb_info_t dvb_dmx_info;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_DWNMIX_LEVEL_DVB,
(void *) &dvb_dmx_info);

```

Errors

- No specific error.

Table 3-37 XA_AACDEC_CONFIG_PARAM_PARSED_DRC_INFO Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_PARSED_DRC_INFO
Description	This parameter expects a pointer to the array of MAX_NUM_CHANNELS (8) elements of type <code>xa_aac_dec_parsed_drc_info_t</code> ; it fills these elements with DRC information parsed from the payload.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to the API structure. <code>i_cmd</code> <code>XA_API_CMD_GET_CONFIG_PARAM</code> <code>i_idx</code> <code>XA_AACDEC_CONFIG_PARAM_PARSED_DRC_INFO</code> <code>pv_value</code> <code>&drc_info</code> – Pointer to MAX_NUM_CHANNELS sized array of structures of type <code>xa_aac_dec_parsed_drc_info_t</code>.
Restrictions	■ This parameter is available only with Multi-Channel libraries. ■ Parsed_drc_info values are updated after every successful frame decoding. ■ The flag <code>drc_info_valid</code> (of structure <code>xa_aac_dec_parsed_drc_info_t</code>) in each element indicates whether the DRC information in that element is valid for a given payload.

Example

```

xa_aac_dec_parsed_drc_info_t drc_info[MAX_NUM_CHANNELS];
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_PARSED_DRC_INFO,
(void *) drc_info);

```

Errors

- No specific error.

Table 3-38 XA_AACDEC_CONFIG_PARAM_PROG_REF_LEVEL Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_PROG_REF_LEVEL
Description	This parameter returns the program reference level if present in the bitstream. This is a 7-bit unsigned value stored in a 32-bit word. The value indicates the PCM level of the decoded output. The dB value corresponding to the output level can be determined as follows: $\text{output_level_in_dB} = -0.25 * \text{prog_ref_level}$.
Actual Parameters	p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_PROG_REF_LEVEL pv_value &prog_ref_level – Pointer to a 32-bit variable containing a 7-bit parameter parsed from the bit-stream.
Restrictions	■ This parameter is available only with the aacmchplus_v2_loas library. ■ If prog_ref_level is unavailable in the input bitstream or until the decoder detects the presence of a program_ref_level parameter in the input stream, then the returned value is 0xffff ffff. In this case, the application should use the default value -31dB.

Example

```

int prog_ref_level;
float prog_ref_level_in_db;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM, XA_AACDEC_CONFIG_PARAM_PROG_REF_LEVEL,
(void *) &prog_ref_level);
if (prog_ref_level == -1) {
    printf("program ref level is not available in the bitstream (default
value is assumed to be -31dB)\n");
}
else {
    prog_ref_level_in_db = -0.25*prog_ref_level;
}

```

Errors

- No specific error.

Table 3-39 XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE
Description	This API returns the operating mode of the decoder for ADTS/LOAS streams. If the decoder is running in frame-by-frame decoding mode, it returns 1. If the decoder is running in streaming mode, it returns 0.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_ DECODE pv_value &enableframeByFrameDecode – Pointer to the operating mode variable. </pre>
Restrictions	None

Example

```

int enableframeByFrameDecode;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_ENABLE_FRAME_BY_FRAME_DECODE
(void *) &enableframeByFrameDecode);

```

Errors

- No specific error.

Table 3-40 XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE
Description	This command gets the MPEG4 amendment4 flag value. Possible return values are 0 and 1. This returns a default value or the value set by the application using XA_API_CMD_SET_CONFIG_PARAM with subcommand XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE is explained in Table 3-19.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE pv_value &bmpeg4_amd4 – Pointer to the value of the MPEG4 amendment4 variable. </pre>
Restrictions	

Example

```

int bmpeg4_amd4;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_MPEG4_AMENDMENT4_ENABLE,
(void *) & bmpeg4_amd4);

```

Errors

- No specific error.

Table 3-41 XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE Subcommand

Subcommand	XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE
Description	This command populates the structure <code>xa_aac_dec_parsed_pce_info_t</code> with data related to channels present in the input stream based on the data from PCE (PCE – program config element) as specified in the MPEG4-AAC audio document ISO/IEC 14496-3 in Section 4.4.1.1 [2]. struct type <code>xa_aac_dec_parsed_pce_info_t</code> is defined in <code>xa_aac_dec_api.h</code>.
Actual Parameters	<pre> p_xa_module_obj api_obj – Pointer to the API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE pv_value &ch_cfg_info – Pointer to the structure containing channels and their arrangement in the stream. </pre>
Restrictions	

Example

```

xa_aac_dec_parsed_pce_info_t ch_cfg_info;
res = (*api_func)(api_obj,
XA_API_CMD_GET_CONFIG_PARAM,
XA_AACDEC_CONFIG_PARAM_CHANNEL_CONFIG_INFO_FROM_PCE,
(void *) &ch_cfg_info);

```

Errors

- No specific error.

3.4 Channel Routing

The *Channel Routing* feature is dependent on the maximum number of decode channels supported (user-specified with the -n option), input channel configuration (from input bitstream), number of input channels (from input bitstream), and channel routing parameter (user-specified with channelOffset-channelIndex pair). The following table captures the *Channel Routing* feature behavior as a function of the parameters mentioned above. Refer to the notes below while inferring the table entries.

Note The input channel configuration (XA_AACDEC_CHANNELMODE) is defined in xa_aac_dec_api.h.

The value for symbols C, L, R, I, r, LFE, Sbl, Sbr, and Cs are defined in Table 3-7.

Table 3-42 Channel Routing Feature Behavior

-n	Number of channels in the stream	Channel Config Enum	Sample offset and supported channel index							
			-0	-1	-2	-3	-4	-5	-6	-7
2	1	1	L/R	IG	FE	FE	FE	FE	FE	FE
	2	4 or 2	L/R	L/R	FE	FE	FE	FE	FE	FE
3	1	1	L/R	IG	IG	FE	FE	FE	FE	FE
	2	4 or 2	L/R	L/R	IG	FE	FE	FE	FE	FE
	3	5	C/L/R	C/L/R	C/L/R	FE	FE	FE	FE	FE
		6	L/R/I	L/R/I	L/R/I	FE	FE	FE	FE	FE
		12	L/R/LFE	L/R/LFE	L/R/LFE	FE	FE	FE	FE	FE
4	1	1	L/R	IG	IG	IG	FE	FE	FE	FE
	2	4 or 2	L/R	L/R	IG	IG	FE	FE	FE	FE
	3	5	C/L/R	C/L/R	C/L/R	IG	FE	FE	FE	FE
		6	L/R/I	L/R/I	L/R/I	IG	FE	FE	FE	FE
		12	L/R/LFE	L/R/LFE	L/R/LFE	IG	FE	FE	FE	FE
	4	7	L/R/I/r	L/R/I/r	L/R/I/r	L/R/I/r	FE	FE	FE	FE
		8	C/L/R/I	C/L/R/I	C/L/R/I	C/L/R/I	FE	FE	FE	FE
		13	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	FE	FE	FE	FE
		14	L/R/I/LFE	L/R/I/LFE	L/R/I/LFE	L/R/I/LFE	FE	FE	FE	FE
5	1	1	L/R	IG	IG	IG	IG	FE	FE	FE
	2	4 or 2	L/R	L/R	IG	IG	IG	FE	FE	FE
	3	5	C/L/R	C/L/R	C/L/R	IG	IG	FE	FE	FE

-n	Number of channels in the stream	Channel Config Enum	Sample offset and supported channel index							
			-0	-1	-2	-3	-4	-5	-6	-7
		6	L/R/I	L/R/I	L/R/I	IG	IG	FE	FE	FE
		12	L/R/LFE	L/R/LFE	L/R/LFE	IG	IG	FE	FE	FE
	4	7	L/R/I/r	L/R/I/r	L/R/I/r	L/R/I/r	IG	FE	FE	FE
		8	C/L/R/I	C/L/R/I	C/L/R/I	C/L/R/I	IG	FE	FE	FE
		13	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	IG	FE	FE	FE
		14	L/R//LFE	L/R//LFE	L/R//LFE	L/R//LFE	IG	FE	FE	FE
	5	9	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	FE	FE	FE
		15	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	FE	FE	FE
		16	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	FE	FE	FE
6	1	1	L/R	IG	IG	IG	IG	IG	FE	FE
	2	4 or 2	L/R	L/R	IG	IG	IG	IG	FE	FE
	3	5	C/L/R	C/L/R	C/L/R	IG	IG	IG	FE	FE
		6	L/R/I	L/R/I	L/R/I	IG	IG	IG	FE	FE
		12	L/R/LFE	L/R/LFE	L/R/LFE	IG	IG	IG	FE	FE
	4	7	L/R/I/r	L/R/I/r	L/R/I/r	L/R/I/r	IG	IG	FE	FE
		8	C/L/R/I	C/L/R/I	C/L/R/I	C/L/R/I	IG	IG	FE	FE
		13	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	IG	IG	FE	FE
		14	L/R//LFE	L/R//LFE	L/R//LFE	L/R//LFE	IG	IG	FE	FE
	5	9	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	C/L/R/I/r	IG	FE	FE
		15	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	L/R/I/r/LFE	IG	FE	FE
		16	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	C/L/R/I/LFE	IG	FE	FE
	6	10	C/L/R/I/r/Cs	C/L/R/I/r/Cs	C/L/R/I/r/Cs	C/L/R/I/r/Cs	C/L/R/I/r/Cs	C/L/R/I/r/Cs	FE	FE

-n	Number of channels in the stream	Channel Config Enum	Sample offset and supported channel index							
			-0	-1	-2	-3	-4	-5	-6	-7
		17	C/L/R/ l/r/LFE	C/L/R//r /LFE	C/L/R//r /LFE	C/L/R// r/LFE	C/L/R// r/LFE	C/L/ R//r/ LFE	FE	FE
7	1	1	L/R	IG	IG	IG	IG	IG	IG	FE
	2	4 or 2	L/R	L/R	IG	IG	IG	IG	IG	FE
	3	5	C/L/R	C/L/R	C/L/R	IG	IG	IG	IG	FE
		6	L/R/I	L/R/I	L/R/I	IG	IG	IG	IG	FE
		12	L/R/ LFE	L/R/LFE	L/R/LFE	IG	IG	IG	IG	FE
	4	7	L/R//r	L/R//r	L/R//r	L/R//r	IG	IG	IG	FE
		8	C/L/R/ I	C/L/R/I	C/L/R/I	C/L/R/I	IG	IG	IG	FE
		13	C/L/R/ LFE	C/L/R/ LFE	C/L/R/ LFE	C/L/R/ LFE	IG	IG	IG	FE
		14	L/R// LFE	L/R//LF E	L/R//LF E	L/R// LFE	IG	IG	IG	FE
	5	9	C/L/R/ l/r	C/L/R//r	C/L/R//r	C/L/R// r	C/L/R// r	IG	IG	FE
		15	L/R//r/ LFE	L/R//r/ LFE	L/R//r/ LFE	L/R//r/ LFE	L/R//r/ LFE	IG	IG	FE
		16	C/L/R/ I/LFE	C/L/R// LFE	C/L/R// LFE	C/L/R// LFE	C/L/R// LFE	IG	IG	FE
	6	10	C/L/R/ l/r/Cs	C/L/R//r /Cs	C/L/R//r /Cs	C/L/R// r/Cs	C/L/R// r/Cs	C/L/ R//r/ Cs	IG	FE
		17	C/L/R/ l/r/LFE	C/L/R//r /LFE	C/L/R//r /LFE	C/L/R// r/LFE	C/L/R// r/LFE	C/L/ R//r/ LFE	IG	FE
	7	11	C/L/R/ l/r/Sbl Sbr	C/L/R//r /SblSbr	C/L/R//r /SblSbr	C/L/R// r/SblSb r	C/L/R// r/SblSb r	C/L/ R//r/ SblS br	C/L/R/ l/r/Sbl Sbr	FE
		18	C/L/R/ l/r/Cs/ LFE	C/L/R//r /Cs/LFE	C/L/R//r /Cs/LFE	C/L/R// r/Cs/LF E	C/L/R// r/Cs/LF E	C/L/ R//r/ Cs/L FE	C/L/R/ l/r/Cs/ LFE	FE
8	1	1	L/R	IG	IG	IG	IG	IG	IG	IG
	2	4 or 2	L/R	L/R	IG	IG	IG	IG	IG	IG
	3	5	C/L/R	C/L/R	C/L/R	IG	IG	IG	IG	IG
		6	L/R/I	L/R/I	L/R/I	IG	IG	IG	IG	IG

-n	Number of channels in the stream	Channel Config Enum	Sample offset and supported channel index							
			-0	-1	-2	-3	-4	-5	-6	-7
	4	12	L/R/LFE	L/R/LFE	L/R/LFE	IG	IG	IG	IG	IG
		7	L/R/l/r	L/R/l/r	L/R/l/r	L/R/l/r	IG	IG	IG	IG
		8	C/L/R/l	C/L/R/l	C/L/R/l	C/L/R/l	IG	IG	IG	IG
		13	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	C/L/R/LFE	IG	IG	IG	IG
	5	14	L/R/l/LFE	L/R/l/LFE	L/R/l/LFE	L/R/l/LFE	IG	IG	IG	IG
		9	C/L/R/l/r	C/L/R/l/r	C/L/R/l/r	C/L/R/l/r	C/L/R/l/r	IG	IG	IG
		15	L/R/l/r/LFE	L/R/l/r/LFE	L/R/l/r/LFE	L/R/l/r/LFE	L/R/l/r/LFE	IG	IG	IG
	6	16	C/L/R/l/LFE	C/L/R/l/LFE	C/L/R/l/LFE	C/L/R/l/LFE	C/L/R/l/LFE	IG	IG	IG
		10	C/L/R/l/r/Cs	C/L/R/l/r/Cs	C/L/R/l/r/Cs	C/L/R/l/r/Cs	C/L/R/l/r/Cs	C/L/R/l/r/Cs	IG	IG
	7	17	C/L/R/l/r/LFE	C/L/R/l/r/LFE	C/L/R/l/r/LFE	C/L/R/l/r/LFE	C/L/R/l/r/LFE	C/L/R/l/r/LFE	IG	IG
		11	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	C/L/R/l/r/SblSbr	IG
		20	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	C/Sbl/Sbr/L/R/l/r	IG
		21	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	C/L/R/l/r/Sbl/Sbr	IG
	8	18	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	C/L/R/l/r/Cs/LFE	IG
		19	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	C/L/R/l/r/SblSbr/LFE	IG

-n	Number of channels in the stream	Channel Config Enum	Sample offset and supported channel index							
			-0	-1	-2	-3	-4	-5	-6	-7
		22	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE	C/Sbl/Sbr/L/R/l/r/LFE
		23	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr	C/L/R/l/r/LFE/Sbl/Sbr

Note Each entry in Table 3-42 mentions valid channel index values for a given sample offset. IG indicates that any channel index value for a given sample offset will be ignored. FE means that any channel index value for a given sample offset will result in a fatal error.

Note The number of channels routed must be equal to the value set by the user with -n option, or else a fatal error is returned.

Note The decoder supports only unique channel routing (That is, each sample offset and each channel index must be unique).

Note If at least one of the channels present in the stream is not routed, then a fatal error is returned.

Note The number of channels written in the output buffer is equal to the default or the user-specified value with -n option.

4. Introduction to the Example Testbench

The supplied testbench consists of the following files:

- Testbench source files (found in `test/src` directory)
 - `xa_aac_dec_error_handler.c`
 - `xa_aac_dec_sample_testbench.c`
- Makefile to build the executable (`test/build`):
 - `makefile_testbench_sample`
- Sample parameter file to run the testbench (`test/build`):
 - `paramfilesimple.txt`

4.1 Making Executable Files

This section describes how to build the decoder testbench and the library.

Build Decoder Testbench

To build the application, follow these steps:

1. In the command prompt, navigate to the `test/build` directory.
2. In the console, enter the following command:

```
xt-make -f makefile_testbench_sample clean <lib>
```

(where `<lib>` is one of `aac`, `aacplus`, `aacplus_v2`, `aacmch`, `aacmchplus`, `aacmchplus_v2`, or `aacmchplus_v2_loas`).

The decoder example testbench `xa_<lib>_dec_test` is built.

Build Decoder Testbench with LOAS/LATM

To build the decoder testbench with LOAS/LATM support for other decoder variants (other than `aacmchplus_v2_loas`, which can be built by step 2 above), follow these steps:

1. In the console, enter the following command:

```
xt-make -f makefile_testbench_sample clean <lib> LINK_LOAS=1
```

(where `<lib>` is one of `aac`, `aacplus`, `aacplus_v2`, `aacmch`, or `aacmchplus`)

The decoder example testbench `xa_<lib>_loas_dec_test` is built.

Build Decoder Library

Note If you have the source code distribution, you must build the `<lib>` library before you can build the testbench.

To build the library, follow these steps.

1. In the command prompt, navigate to the `build` directory.
2. Enter the following command:

```
$xt-make clean <lib> install
```

If `<lib>` is one of `aacmchplus_v2` or `aacmchplus_v2_loas`, the `xa_<lib>_dec.a` library is built and copied to the `lib` directory.

If `<lib>` is one of `aac`, `aacplus`, `aacplus_v2`, `aacmch`, `aacmchplus`, the `xa_<lib>_dec.a` and `xa_<lib>_loas_dec.a` libraries are built and copied to the `lib` directory.

Select LOAS Library

The `aac`, `aacplus`, `aacplus_v2`, `aacmch`, and `aacmchplus` object `.xws` packages will use theNon-LOAS library by default. To switch to the LOAS library, follow these steps (here, `aac` is used as an example):

1. In the **Project Explorer** area of Xplorer, click the triangle to the left of `libxa_aac_dec` to expand the folder; then right-click `ulibxa_aac_dec/lib/xa_aac_dec_loas.a` and select **Unmanaged Binary Info**. Then select the appropriate config (for example, `AE_HiFi3_LE5`) and click **OK**.
2. Right-click `testxa_aac_dec` and select **Library Dependencies...**, `ulibxa_aac_dec/lib/xa_aac_dec_loas.a (AE_HiFi3_LE5)` is now shown in the Available libraries area. Select this file and click **Add**. The file is now shown in the **Selected Libraries** area.
3. Select `ulibxa_aac_dec/lib/xa_aac_dec.a (AE_HiFi3_LE5)` and click **Remove**, then **Apply** and **OK**.

Build LOAS Library

To build the `loas` library with the `aac`, `aacplus`, `aacplus_v2`, `aacmch`, or `aacmchplus` source `xws` package, follow the steps below (again using `aac` as an example).

1. In the **Project Explorer** area of Xplorer, click the triangle to the left of `libxa_aac_dec` to expand the folder; then double-click `Makefile.include` to open the file. Change the line `OBJS = xxx` to `OBJS = algo/transport/src/loas.o xxx` and click **Save**. Next, browse to `algo/transport/src/loas.c` and right-click to select **Build->Include**.
2. Right-click `libxa_aac_dec` and select **Build Properties....** Select **CommonTarget** as the target.

- a. Click to select the **Symbols** tab; then click the **Add symbol** icon (green plus sign) to add two symbols, AACLOAS_SUPPORT and LOAS_SUBFRAME_SUPPORT.
 - b. From the folder structure on the left, browse to algo/transport/src/loas.c left. Select the **Add compiler** tab and type -Os -x c++ in the Local: area.
 - c. Verify that the new symbols and options are shown in the All Options area at the bottom.
3. Select P:libxa_aac_dec and click **Build Active**.
Note that the library is still called libxa_aac_dec.a, but it now supports LOAS.

4.2 Usage

You can run the sample application using the direct command line options or with a parameter file.

The command line usage is as follows:

```
xt-run xa_<lib>_dec_test -ifile:<infile> -ofile:<outfile>
      [-b<bsformat>]
      [-d<bdownsample>]
      [-f<to_stereo>]
      [-m<downmix>]
      [-n<outnchans>]
      [-p<extsr>]
      [-w<pcm_wdsz>]
      [-x<sbr_signaling>]
      [-0..7<ch>]
      [-a<amendment4>]
      [-rst<int>]
```

The following options are available for a LOAS build:

```
[--d<drc>]
[--dC<drc_compress>]
[--dB<drc_boost>]
[--p<prl>]
[--pL<target_level>]
```

Where:

<code><lib></code>	One of	aac, aacplus, aacplus_v2, aacmch, aacmchplus, aacmchplus_v2 aacmchplus_v2_loas, aac_loas, aacplus_loas, aacplus_v2_loas,
--------------------------	--------	---

	aacmch_loas, aacmchplus_loas
<infile>	Name of the AAC input file.
<outfile>	Name of the output “.wav” file.
<bsformat>	Bitstream format (-b raw, adif, adts, latm, loas)
<bdownsample>	Enables (1) or disables (0) downsampled SBR mode (disabled by default).
<to_stereo>	Enables (1) or disables (0) duplication of mono output to interleaved stereo (enabled by default).
<downmix>	Enables (1) or disables (0) SBR mono downmix (disabled by default).
<outnchans>	Maximum number of channels to be decoded (2 to 8).
<extsr>	The sample rate of a raw bitstream (-p 44100).
<pcm_wdsz>	Output PCM word size – 16 or 24 (default).
<sbr_signaling>	Disables (0), Enables (1), or turn SBR processing in auto mode (2) (auto by default).
<ch>	Input channel (L, C, R, I, r, Sbl, Sbr, LFE) to route to an arbitrary output buffer offset (0 through 7): -0L -1C -2R -3l -4r -5Sbl -6Sbr -7LFE.
<drc>	Enables (1) /Disables (0) flag for applying DRC (default = 0) .
<drc_compress>	DRC Compression factor between 0.0 to 100.0 (default value is 0.0) .
<drc_boost>	DRC Boost factor between 0.0 to 100.0 (default value is 0.0) .
<prl>	Enables (1) /Disables flag (0) for applying Program Ref level (default value is 0) .
<target_level>	Target Level value between 0 to 127, indicating the level in dB as $-0.25 * \text{target_level}$ (default value is 124) .
<amendment4>	Enables (1) /Disables (0) flag for applying MPEG4 amendment4 changes (default = 1) .
<int>	Resets internal decoder states in case of sync loss error (0 - disable, 1 - enable by default).

For a full description of their usage, refer to the parameter definitions in Section 3.3. Note that the space between the option name and the value is optional.

If command-line arguments are not provided, the application reads the commands from the paramfilesimple.txt parameter file.

The following is the syntax for writing the `paramfilesimple.txt` file:

```
@Start

@Input_path <path to be appended to all input files>
@Output_path <path to be appended to all output files>
<command line 1>
<command line 2>
....
@Stop
```

The AACMCH Decoder can be run for multiple test files using different command lines. The syntax for command lines in the parameter file is the same as the syntax for specifying options on the command line to the testbench program.

Note	All the <code>@<command></code> s should be at the first column of a line except the <code>@New_line</code> command.
-------------	--

Note	All the <code>@<command></code> s are case sensitive. If the command line in the parameter file has to be divided into two parts on two different lines, use the <code>@New_line</code> command, as shown in the following example.
-------------	---

```
<command line part 1> @New_line
```

```
<command line part 2>.
```

Note	Blank lines will be ignored.
-------------	------------------------------

Note	Individual lines can be commented out using <code>"//"</code> at the beginning of the line.
-------------	---

5. References

- [1] *ISO/IEC 13818-7 Information technology -- Generic coding of moving pictures and associated audio information -- Part 7: Advanced Audio Coding (AAC). (MPEG-2)*
- [2] *ISO/IEC 14496-3: Information technology -- Coding of audio-visual objects -- Part 3: Audio (MPEG-4)*
- [3] *ISO/IEC 14496-3:2001/Amd1, Bandwidth Extension (MPEG-4)*
- [4] *ISO/IEC 14496-3:2001/Amd2, Parametric Audio for High Quality Audio (MPEG-4)*
- [5] *Coding Technologies' aacPlus Fixed Point Firmware Reference Decoder v4.0.3*